

Five agent architectures, one filthy room

A complete walkthrough of the vacuum-world notebook — **every line of code explained**, every output shown as it actually ran, and full worked solutions to all four exercises. Read it top to bottom and you will know not just what each agent does, but why the architectures stop being interchangeable the moment the world gets harder.

BASED ON Tutorial_1_Agents_Hands_On.ipynb	CODE Python 3.11, no dependencies	OUTPUTS Executed, not transcribed
COVERS 9 cells + 4 solutions		

00 The one idea to hold on to

Everything in this notebook is a variation on a single loop. The agent *senses*, *decides*, and *acts*; the world changes; it senses again.

An **agent** is anything that perceives its environment through **sensors** and acts on it through **actuators**. What it does is defined by an **agent function** that maps a percept sequence to an action. That function is an abstraction; the **agent program** is the concrete code that implements it. The five architectures in this tutorial are five different ways of writing that program.

THE SPLIT THAT MATTERS

The notebook keeps the **environment** and the **agent** in separate objects that share nothing but a percept and an action string. The environment knows the whole world; the

agent knows only what `percept()` hands it. Every interesting result later comes from that gap.

What each architecture adds

ARCHITECTURE	WHAT IT ADDS	THE QUESTION IT CAN ANSWER
Simple reflex	Nothing — rules on the current percept only	“What is under me right now?”
Model-based reflex	Internal state: memory of what it has seen	“What is the world like, including where I am not?”
Goal-based	An explicit goal and a plan to reach it	“What sequence gets me to the state I want?”
Utility-based	A scoring function over competing outcomes	“Which of these imperfect options is best?”
Learning	A feedback loop that edits its own policy	“What worked last time?”

They are cumulative in capability but not in cost. The whole payoff of the exercises at the end is discovering that more capability is not automatically better — each addition brings its own failure mode.

01 The Python you need, in one section

If you are new to Python, read this once before the code sections. Everything the notebook uses appears here with a tiny example. Each line-by-line annotation later also carries an orange **Python** box explaining the syntax on that line, so you can come back here only when you want the fuller version.

Indentation is not decoration

Most languages group code with curly braces. Python groups it with **indentation** — the spaces at the start of a line are part of the meaning. Lines indented under an `if` run only when it is true.

```

1 if status == "Dirty":
2     return "Suck"           # indented: only runs when the condition is true
3 return "Right"             # not indented: runs whenever we get this far

```

Note `==` to *compare* and a single `=` to *assign*. Mixing them up is the most common beginner error in any language.

The four containers

TYPE	LOOKS LIKE	WHAT IT IS
string	<code>"Dirty"</code>	Text. Single or double quotes, your choice.
list	<code>["A", "B"]</code>	An ordered, changeable sequence. Add with <code>.append(x)</code> .
tuple	<code>("A", "Dirty")</code>	Like a list but <i>cannot be changed</i> after creation. Used here for percepts.
dict	<code>{"A": "Dirty"}</code>	A lookup table of key → value. This is how the world's state is stored.

You read a dict with square brackets, and write to it the same way:

```

1 status = {"A": "Dirty", "B": "Clean"}
2 print(status["A"])           # -> Dirty           (read)
3 status["A"] = "Clean"       #                   (write, or create if missing)

```

Indexing counts from **zero**, and negative numbers count from the end:

```

1 row = ["A", "B", "C", "D"]
2 row[0]    # -> "A"    the first item
3 row[1]    # -> "B"    the second
4 row[-1]   # -> "D"    the last item

```

Unpacking

You will see this on almost every agent's first line. If the right side has several values, the left side can name them all at once:

```
1 percept = ("A", "Dirty")
2 location, status = percept      # location is now "A", status is now "Dirty"
```

When you do not need one of the values, the convention is to name it `_`, which means “ignore this”:

```
1 _, _, trace = run(agent)      # keep only the third value
```

Functions

```
1 def run(agent_program, steps=6, seed=42):
2     ...
3     return env.performance
```

- `def` starts a function; the indented block below is its body.
- `steps=6` is a **default**: callers can leave it out and get 6.
- `return` hands a value back *and exits immediately* — nothing after it runs.
- Returning several comma-separated values actually returns one tuple, which the caller can unpack.

Functions are values (this is the big one)

In Python a function can be stored in a variable, put in a list, and passed to another function. The presence or absence of parentheses decides everything:

```
1 program          # the function itself, not run
2 program(percept) # RUN it, with percept as input
```

That distinction is why `run(simple_reflex_agent())` has two sets of brackets: the inner pair builds an agent, and `run` calls it later, once per step.

A function that returns a function (a “closure”)

Every agent in this notebook is written this way, so it is worth slowing down on:

```

1  def make_counter():
2      count = {"n": 0}           # lives in the OUTER function
3      def step():                # an inner function
4          count["n"] += 1        # it can see and change count
5          return count["n"]
6      return step                # hand back the inner function
7
8  c = make_counter()
9  c()    # -> 1
10 c()    # -> 2    the count SURVIVED between calls

```

The inner function keeps access to the outer function's variables even after the outer function has finished. That is a **closure**, and it is how these agents remember things without being classes. Call `make_counter()` twice and you get two independent counters — which is why the comparison loop stores factories and calls each one fresh.

WHY THE STATE IS WRAPPED IN A DICT

Inside a nested function, `count = count + 1` would create a brand-new local variable instead of changing the outer one. Two ways round it: write `nonlocal count` first, or store the value inside a dict and write `count["n"] += 1`, which modifies rather than re-assigns. The notebook uses both — that is why you keep seeing single values wrapped in dictionaries.

Conditional expressions

An `if` squeezed into a single value. Read it left to right: *this* if *condition*, otherwise *that*.

```

1  return "Right" if location == "A" else "Left"
2
3  # exactly the same thing, written out:
4  if location == "A":
5      return "Right"
6  else:
7      return "Left"

```

Comprehensions

A one-line way to build a list or dict from a loop.

```

1  # dict comprehension: one entry per location
2  {loc: "Dirty" for loc in ["A", "B"]}      # -> {"A": "Dirty", "B": "Dirty"}
3
4  # list comprehension
5  [x * 2 for x in [1, 2, 3]]                # -> [2, 4, 6]
6
7  # the same shape without brackets is a "generator" - it produces values
8  # one at a time, which is all that all() and sum() need
9  all(v == "Clean" for v in status.values()) # True only if EVERY one matches
10 sum(1 for v in status.values() if v == "Clean") # counts the clean ones

```

max() with a key

The utility and learning agents both turn on this one function.

```

1  actions = ["Suck", "Left", "Right", "NoOp"]
2  best = max(actions, key=lambda a: utility(a))

```

- `lambda a: ...` is a small unnamed function, taking `a` as its input.
- `max(items, key=f)` runs `f` on every item and returns the **item** with the highest score — not the score itself.
- On a tie, the **first** item in the list wins. That silent tie-break matters more than once in this notebook.

Classes

Only the environment uses a class. A class is a blueprint; an *instance* is one thing built from it.

```

1  class VacuumEnvironment:
2      def __init__(self, seed=42):          # runs automatically when you create one
3          self.performance = 0              # self.x = data belonging to THIS instance
4
5      def percept(self):                    # a method: a function inside a class
6          return self.agent_location
7
8  env = VacuumEnvironment(seed=1)          # create an instance
9  env.percept()                           # call a method - you never pass self

```

- `self` is the instance itself. It is always the first parameter, and Python fills it in for you when you call the method.
- Names with `self.` in front survive after the method ends. Plain local names do not.
- `__init__`'s double underscores mark it as one of Python's special method names.

Two further pieces appear in the exercise solutions:

```
1 class StochasticVacuumEnvironment(VacuumEnvironment): # inheritance
2     def execute(self, action):
3         super().execute(action) # run the ORIGINAL version first
4         ... # then add new behaviour
5
6     @property # a decorator: lets you write env.agent_location
7     def agent_location(self): # with NO parentheses, though it is really a method
8         return self.locations[self.index]
```

f-strings and format specs

The `f` before the quote lets you drop variables into text with `{}`. What comes after a colon controls the padding, which is what makes all the output tables line up.

```
1 name, perf = "Simple Reflex", 5
2
3 f"{name}" # -> 'Simple Reflex'
4 f"{name:22s}" # -> 'Simple Reflex' string, padded to 22, left-aligned
5 f"{perf:>12d}" # -> '5' whole number, padded to 12, RIGHT-aligned
6 f"{perf:+6.1f}" # -> '+5.0' always show sign, 6 wide, 1 decimal
7 f"{0.73:.0%}" # -> '73%' fraction as a percentage
```

A few smaller pieces that appear throughout:

YOU'LL SEE	IT MEANS
<code>x += 1</code>	Add to what is already there. Also <code>-=</code> for subtract.
<code>#</code>	A comment. Python ignores the rest of the line.
<code>"""..."""</code>	A string that can span lines. As the first line of a function it is a <i>docstring</i> (documentation).
<code>None</code>	"Nothing here yet". Test it with <code>is None</code> , never <code>== None</code> .
<code>and</code> / <code>or</code> / <code>not</code>	Combine or flip true/false conditions.
<code>in</code>	Membership: <code>action in ("Left", "Right")</code> .
<code>**</code>	Powers, not multiplication. <code>0.6 ** 3</code> is $0.6 \times 0.6 \times 0.6$.
<code>pass</code>	Do nothing. Used where Python demands an indented block but you want none.
<code>raise</code>	Stop with an error.
<code>"</code> <code>".join(items)</code>	Glue a sequence of strings together with a space between.
<code>True</code> is <code>1</code>	Booleans are numbers, so <code>count += is_clean()</code> counts how many times it was true.

YOU DO NOT NEED TO MEMORISE THIS

Skim it now, start reading the code, and come back when a line looks strange. Every annotation in the sections that follow has its own Python box, so the explanation you need is always right next to the line that needs it.

02 The environment (the world the agent lives in)

This is the longest cell in the notebook and the most important. Read it as a contract: it defines exactly what the agent is allowed to know and exactly what it is allowed to do.

Cell 1 – the vacuum world


```

1  import random
2  from copy import deepcopy
3
4  LOCATIONS = ["A", "B"]
5
6  class VacuumEnvironment:
7      def __init__(self, dirt_prob=0.5, seed=42):
8          rng = random.Random(seed)
9          self.status = {loc: ("Dirty" if rng.random() < dirt_prob else "Clean") for loc in LOCATIONS}
10         self.agent_location = rng.choice(LOCATIONS)
11         self.performance = 0          # scorecard: +10 per successful clean, -1 per move
12         self.history = []             # (location, action, world_state) trace
13
14     def percept(self):
15         '''Sensors: the agent perceives only its current location and its status'''
16         return (self.agent_location, self.status[self.agent_location])
17
18     def execute(self, action):
19         '''Actuators: apply the chosen action to the environment.'''
20         loc = self.agent_location
21         if action == "Suck":
22             if self.status[loc] == "Dirty":
23                 self.status[loc] = "Clean"
24                 self.performance += 10
25             elif action == "Left":
26                 self.agent_location = "A"
27                 self.performance -= 1
28             elif action == "Right":
29                 self.agent_location = "B"
30                 self.performance -= 1
31             elif action == "NoOp":
32                 pass
33             else:
34                 raise ValueError(f"Unknown action: {action}")
35
36         self.history.append((loc, action, deepcopy(self.status)))
37         return self.percept()
38
39     def is_clean(self):
40         return all(v == "Clean" for v in self.status.values())
41
42
43 # Quick sanity check
44 env = VacuumEnvironment(seed=1)

```

```
45 print("Initial world state:", env.status)
46 print("Initial percept:", env.percept())
```

L1

The only import the core tutorial needs. `random` generates the starting world — and later, the learning agent's exploration.

PYTHON

`import random` loads Python's built-in random-numbers module. After this line you write `random.something()` to use anything inside it.

L2

`deepcopy` is imported for one job, on line 36. It is what makes the history a genuine *trace* instead of the same dictionary repeated.

PYTHON

`from copy import deepcopy` takes just *one* name out of a module, so you write `deepcopy(x)` instead of `copy.deepcopy(x)`. Two ways of importing; both are normal.

L4

The world is two squares, `A` and `B`. A module-level constant so both the environment and the agents refer to the same world. Exercise 3 changes this line and discovers how much depends on it.

PYTHON

A **list** is an ordered collection written in square brackets. `LOCATIONS` holds two text values (**strings**). ALL-CAPS is a convention meaning “this is a constant, don't change it” — Python does not enforce that, it is just a message to humans.

L6

The environment is its own class. The agent is not a method on it — that separation is the whole design.

PYTHON

`class` defines a new *type* of thing. Think of it as a blueprint: it says what data every vacuum world holds and what it can do. Making one is called *creating an instance*, which line 44 does with `VacuumEnvironment(seed=1)`.

L7

`dirt_prob` is the chance each square starts dirty. `seed` makes a run reproducible, which is what lets you compare architectures fairly.

PYTHON

`__init__` is the **constructor** — Python runs it automatically when you create an instance. The double underscores mark it as special. `self` is the instance being built, and it is *always* the first parameter of a method; you never pass it in yourself. `dirt_prob=0.5` is a **default value**: callers may leave it out.

L8

A *private* random generator, not the global `random` module. So the world's layout cannot be shifted by anything else in the notebook calling `random` — which matters, because the learning agent does exactly that.

PYTHON

`random.Random(seed)` creates a private random-number generator object and stores it in a plain local variable `rng`. Because it has no `self.` in front, it disappears when `__init__` finishes — it is only needed during setup.

L9

A dict comprehension building `{'A': 'Dirty' | 'Clean', 'B': ...}`. Each square is dirty with probability `dirt_prob`. This dictionary is the world.

PYTHON

A **dict** (dictionary) maps keys to values, like a lookup table. This is a **dict comprehension**: `{key: value for item in collection}` builds one in a single line. Read it right-to-left: for each `loc` in `LOCATIONS`, make an entry. The value uses a **conditional expression**: `A if condition else B` gives `A` when the condition holds, otherwise `B`. Anything stored on `self.` survives after the method ends.

L10

The agent's starting square, also drawn from the seeded generator.

PYTHON

`rng.choice(...)` picks one random item from a list.

L11

The scorecard, starting at zero. The comment states the whole performance measure: **+10 per successful clean, -1 per move**. That ratio is what makes travelling worthwhile — and Exercise 4 is about what happens when you change it.

PYTHON

`= 0` creates the attribute and sets its starting value. Everything after `#` on a line is a **comment** — Python ignores it completely.

L12

An empty list that will collect `(location, action, world_state)` triples so you can inspect behaviour afterwards.

PYTHON

`[]` is an empty list, ready to be filled later with `.append(...)`.

L14–16

The sensors. The single most consequential method here. It returns a tuple of the agent's *current location* and the status of *that square only*. It cannot see the other square. Everything the agent does not know, it does not know because of this one line.

PYTHON

A **method** is just a function defined inside a class. The text in triple quotes on line 15 is a **docstring** — documentation, not code. Line 16 returns a **tuple**: several values grouped with parentheses and commas. `self.status[self.agent_location]` is a dict lookup: get the value stored under that key.

L18–19

The actuators. Takes an action string and applies its effect to the world.

PYTHON

This method takes one real argument besides `self`. Calling it looks like `env.execute("Suck")` — Python fills in `self` for you.

L20

Cache the location *before* the action potentially moves the agent. Line 36 needs the square the action was taken *from*.

PYTHON

Copies the current value into a short local name. Since strings cannot be changed in place, `loc` keeps the old value even after line 26 changes `self.agent_location`.

L21–24

`Suck` only scores when the square was actually dirty. Sucking a clean square is silently free: no reward, and no penalty either. Harmless here — but that free no-op is exactly the loophole Exercise 1 catches the utility agent exploiting.

PYTHON

`==` asks “are these equal?” (one `=` assigns, two compare — a classic beginner trap). **Indentation is how Python groups code:** line 22 is inside line 21's `if`, and lines 23–24 are inside line 22's. `+=` means “add to what is already there”, so `self.performance += 10` is short for `self.performance = self.performance + 10`.

L25–27

`Left` sets the location to `"A"` — note this is **absolute, not relative**. In a two-square world “go left” and “go to A” are the same thing. That coincidence hides a real assumption, and Exercises 2 and 3 both turn on it. Cost: `-1`.

PYTHON

`elif` is short for “else if”. Python checks each branch in order and runs only the *first* one that matches, then skips the rest. `-=` subtracts in place.

L28–30

`Right` is the mirror image: go to `B`, pay `-1`.

PYTHON

Another `elif` branch, same shape as the one above it.

L31–32

`NoOp` changes nothing and costs nothing. It is the only free action, which makes it the baseline every utility comparison is measured against.

PYTHON

`pass` means “do nothing here”. Python requires *something* indented under an `if`, so `pass` is the official way to write a deliberately empty block.

L33–34

Anything else is a programming error, raised loudly. A cheap contract check that stops a typo in an agent from silently becoming a no-op.

PYTHON

`else` catches everything the branches above missed. `raise` throws an **exception**, stopping the program with an error. The `f` before the quotes makes it an **f-string**: anything in `{curly braces}` is replaced by that variable's value.

L36

Append the step to the trace. The `deepcopy` matters: `self.status` is mutated in place, so storing it directly would leave every entry pointing at the same dict and the trace would show the final state six times over.

PYTHON

`.append(...)` adds one item to the end of a list. Note the double parentheses: the outer pair belongs to the call, the inner pair builds the tuple being added.

L37

Return the fresh percept as a convenience. The runner ignores it and calls `percept()` itself, so this is belt-and-braces.

PYTHON

A method may call another method on the same object via `self.`

L39–40

A god's-eye check on whether the whole world is clean. Note carefully: **this is a method on the environment, not the agent**. The agent can never call it — it would be cheating. Knowing when to stop is something each architecture must earn for itself.

PYTHON

`all(...)` returns `True` only if every item is true. Inside it is a **generator expression** — the same shape as a comprehension but without brackets — producing one `True / False` per square. `.values()` gives just the values of a dict, ignoring the keys.

L43–46

A sanity check. Build a world with `seed=1`, print the true state, then print what the agent would actually perceive.

PYTHON

Creating an instance looks like calling the class: `VacuumEnvironment(seed=1)`. Writing `seed=1` is a **keyword argument** — naming the parameter instead of relying on position. `print` with several comma-separated values prints them with spaces between.

OUTPUT

```
Initial world state: {'A': 'Dirty', 'B': 'Clean'}
Initial percept: ('A', 'Dirty')
```

Look at those two output lines together. The world is   agent at A — but the percept is just `('A', 'Dirty')`. The fact that `B` is already clean exists in the

environment and is invisible to the agent. Hold that thought; it is the entire plot.

03 **PEAS: specifying the task before you code it**

PEAS — Performance, Environment, Actuators, Sensors — is a checklist for pinning down a task precisely enough to build for it. Here it is written out as a plain dictionary.

Cell 2 – the task specification

```
1 vacuum_world_peas = {
2     "Performance": "Reward for clean squares, small penalty per move",
3     "Environment": "Two locations (A, B), each Dirty or Clean",
4     "Actuators": "Suck, Left, Right, NoOp",
5     "Sensors": "Current location, dirt status of current location",
6 }
7
8 for k, v in vacuum_world_peas.items():
9     print(f"{k:12s}: {v}")
```

L1–6

A plain dictionary, one key per PEAS letter. There is no clever machinery here on purpose: the value is in being forced to *write the four answers down* before writing agent code.

PYTHON

A dict written out in full, one "key": "value" pair per line. The trailing comma after the last pair is legal and normal in Python — it makes adding another line a clean one-line diff.

L2

Performance: reward for clean squares, small penalty per move. Note that this is a description of lines 11, 24, 27 and 30 of the previous cell — the spec and the code have to agree, and nothing enforces it but you.

L3

Environment: two locations, each dirty or clean. Small, static, fully observable at each square, and episodic. Every one of those four properties is deliberately broken by one of the exercises.

L4

Actuators: the four legal action strings, matching the branches of `execute()` exactly.

L5

Sensors: current location and dirt status *of the current location*. The phrasing carries the limitation.

L8–9

Print each pair. `f"{k:12s}"` left-pads the key to 12 characters so the colons line up into a column.

PYTHON

`.items()` loops over a dict giving both key and value at once, so `for k, v in ...` unpacks each pair into two variables. Inside an f-string, the part after the colon is a **format spec**: `{k:12s}` means "this is a string, pad it to 12 characters".

OUTPUT

```
Performance : Reward for clean squares, small penalty per move
Environment  : Two locations (A, B), each Dirty or Clean
Actuators    : Suck, Left, Right, NoOp
Sensors      : Current location, dirt status of current location
```

WHY THIS CELL EARNS ITS PLACE

Every failure in the exercise section is a PEAS assumption that stopped being true. Dirt that reappears breaks *Environment*. Hiding the location breaks *Sensors*. A deadline

breaks *Performance*. Writing PEAS down is what makes those failures legible instead of mysterious.

04 The run loop (the harness everything is scored by)

Seven lines, and it is the sense–decide–act cycle in its plainest possible form. Every score in this tutorial comes out of this function.

Cell 4 – the harness

```
1 def run(agent_program, steps=6, seed=42):
2     env = VacuumEnvironment(seed=seed)
3     for _ in range(steps):
4         p = env.percept()
5         a = agent_program(p)
6         env.execute(a)
7     return env.performance, env.is_clean(), env.history
```

L1	Takes an <i>agent program</i> — a function from percept to action — plus a step budget and a seed. Because the agent is just a function, every architecture can be scored by the identical harness. PYTHON A plain function, not in a class, so no <code>self</code>. Note that <code>agent_program</code> is a function being passed as an argument — in Python functions are ordinary values you can store in variables and hand around.
L2	A fresh environment per run. Passing the same seed to every agent guarantees they all face the same starting world, which is what makes the comparison table meaningful. PYTHON Creating a new environment instance. <code>seed=seed</code> looks odd but is normal: the left <code>seed</code> is the parameter name in <code>VacuumEnvironment</code>, the right one is this function's variable.
L3	Run for a fixed number of steps. <code>_</code> signals the counter is unused. Note there is no termination check — the loop does not stop when the world is clean. That is deliberate: it means idling has to be a choice the agent makes, not a favour the harness does for it. PYTHON <code>range(steps)</code> produces the numbers 0, 1, 2, ... up to but not including <code>steps</code>. Naming the loop variable <code>_</code> is a convention meaning “I must name it, but I never use it”.
L4	Sense. Ask the environment what the agent can currently perceive. PYTHON Calling a method and storing what it returns.
L5	Decide. Hand that percept to the agent program and receive an action string. This line is the entire agent interface. PYTHON <code>agent_program(p)</code> <i>calls</i> the function that was passed in. This is the payoff of functions-as-values: the harness has no idea which agent it is running.
L6	Act. Apply the action to the environment, which updates the world, the score and the trace. PYTHON A call whose return value is ignored — you only want its side effect on the environment.
L7	Return final score, whether the world ended up clean, and the full trace.

PYTHON

Returning several values separated by commas actually returns one tuple. The caller can then unpack it into three variables in one go, as line 1 of the next cell does.

A DETAIL WORTH NOTICING

Six steps is a tight budget for a world that needs at most one move and two sucks. That tightness is why `NoOp` is worth points: an agent that keeps pacing after the job is done burns -1 per step, and there are only six steps to spend.

05 Architecture 1 — Simple reflex

Condition–action rules on the current percept. No memory of anything.

Cell 3 – simple reflex agent

```
1 def simple_reflex_agent():
2     '''Rule: if the current square is dirty, clean it; otherwise move to the other
3     def program(percept):
4         location, status = percept
5         if status == "Dirty":
6             return "Suck"
7         elif location == "A":
8             return "Right"
9         else:
10            return "Left"
11    return program
```

L1

A *factory*: calling `simple_reflex_agent()` returns a fresh program. Every architecture in the notebook follows this shape, so the comparison loop can call `factory()` uniformly and each agent starts with clean internal state.

PYTHON

A function that *returns another function*. This is a **closure**, and it takes a moment to get used to: `simple_reflex_agent` is not the agent, it *builds* an agent and hands it back.

L2

The docstring is the agent function in English: dirty → clean it, otherwise move.

PYTHON

A docstring: the first string inside a function is documentation. Triple quotes let it span lines.

L3

The inner function is the actual agent program. Its only input is `percept` — no history, no world access.

PYTHON

A `def` inside another `def`. The inner function is defined fresh each time the outer one runs.

L4

Unpack the tuple from `percept()` into the two things the agent knows.

PYTHON

Tuple unpacking. `percept` holds a two-item tuple like `('A', 'Dirty')`; this line splits it into two named variables in one step. Equivalent to `location = percept[0]` then `status = percept[1]`, but much easier to read.

L5–6

Rule one: if the square under me is dirty, suck. Immediate and always correct.

PYTHON

`return` immediately exits the function, so nothing below it runs on this call.

L7–8

Rule two: a clean square at `A` means go right.

PYTHON

Reached only when the first condition was false.

L9–10

Rule three: otherwise (clean, at `B`) go left.

PYTHON

`else` with no condition: the catch-all.

L11

Return the program. Nothing is captured in the closure — **there is no state to capture**. That absence is the definition of this architecture.

PYTHON

No parentheses here. `return program` hands back the function itself; `return program()` would call it and return its result instead. That one difference is the whole trick.

Watching it run

The trace cell runs the agent and prints one line per step.

Cell 5 – trace, seed=3

```
1 _, _, trace = run(simple_reflex_agent(), seed=3)
2 for step, (loc, action, state) in enumerate(trace, 1):
3     print(f"Step {step}: at {loc} -> action={action:6s} -> world={state}")
```

L1

Run the agent and keep only the trace; `_, _` discards score and cleanliness. Note `seed=3` — a different world from the default.

PYTHON

Unpacking three returned values, using `_` twice for the two being thrown away. Python allows repeating `_` like this.

L2

`enumerate(trace, 1)` numbers the steps from 1, and each entry unpacks into the triple stored on line 36 of the environment.

PYTHON

`enumerate(trace, 1)` pairs each item with a counter starting at 1. The `for step, (loc, action, state) in ...` unpacks two levels at once: first the counter and the item, then the item's own three parts — the parentheses mark that nested step.

L3

`{action:6s}` pads the action to six characters so the arrows line up into a readable column.

PYTHON

One f-string with several `{}` slots. `{action:6s}` pads to six characters so the columns align.

OUTPUT

```
Step 1: at B -> action=Left    -> world={'A': 'Dirty', 'B': 'Clean'}
Step 2: at A -> action=Suck     -> world={'A': 'Clean', 'B': 'Clean'}
Step 3: at A -> action=Right   -> world={'A': 'Clean', 'B': 'Clean'}
Step 4: at B -> action=Left    -> world={'A': 'Clean', 'B': 'Clean'}
Step 5: at A -> action=Right   -> world={'A': 'Clean', 'B': 'Clean'}
Step 6: at B -> action=Left    -> world={'A': 'Clean', 'B': 'Clean'}
```

Step by step, with the world drawn after each action:

- 1 Starts at **B**, which is clean, so the rule says go left.  agent at B
- 2 Now at **A**, which is dirty → **Suck**. The world is now spotless.
 agent at A
- 3 At **A**, clean → the rule says go right. Cost: -1.
- 4 At **B**, clean → go left. Cost: -1.
- 5 ...and it paces back and forth until the step budget runs out.

THE FAILURE

From step 3 onward this agent is *doing damage* — four wasted moves, -4 points, in a world that has been clean since step 2. It cannot stop, and the reason is precise: stopping requires knowing that *both* squares are clean, and it can only ever see one.
Final score: **+10 - 5 = 5**.

06 Architecture 2 — Model-based reflex

The same reflex rules, plus one dictionary of memory. That dictionary is worth four points.

Cell 6 — model-based reflex agent

```
1 def model_based_reflex_agent():
2     model = {"A": None, "B": None}    # internal state: what we believe about each
3
4     def program(percept):
5         location, status = percept
6         model[location] = status        # update our internal model with what we
7         if status == "Dirty":
8             return "Suck"
9         if model["A"] == "Clean" and model["B"] == "Clean":
10            return "NoOp"                # we believe the job is done -
11            return "Right" if location == "A" else "Left"
12    return program
```

L1

Same factory shape as before.

PYTHON

The same factory-returning-a-closure shape as the previous agent. Getting comfortable with “a function that returns a function” is most of the work in reading this notebook.

L2

The internal state. One dict holding what the agent believes about each square, starting at `None` for “never looked”. Because it lives in the enclosing function, it persists across calls to `program` — this is a closure doing the work an object would do.

PYTHON

A dict with `None` as the starting value. `None` is Python's “nothing here yet” value — deliberately different from `0`, `False`, or an empty string.

L4–5

The program and the same unpacking as before.

PYTHON

The inner function again, with the same unpacking as before.

L6

Update the model. Record what was just observed, keyed by location. The `None` for the other square is what stops the agent claiming knowledge it does not have.

PYTHON

Assigning to a dict key either updates it or creates it. Crucially, `model` belongs to the *outer* function — the inner one can read and modify it, and it survives between calls. That is what makes this memory rather than a fresh start each time.

L7–8

Unchanged reflex rule: dirt under me gets sucked immediately.

PYTHON

Standard early return.

L9–10

The new capability. If the model says both squares are clean, emit `NoOp` and stop burning move penalties. This condition is literally inexpressible for the previous agent — it mentions a square the agent is not standing on.

PYTHON

`and` is true only when both sides are. Python also short-circuits: if the left side is false it never bothers evaluating the right.

L11

Otherwise fall back to the reflex move. Reaching this line means at least one square is still `None` or `Dirty`, so there is a reason to travel.

PYTHON

A conditional expression used directly as the return value: return A if condition else B . Same meaning as a four-line if/else, just compact.

L12

Return the program — this time *with* `model` captured in its closure.

PYTHON

Return the inner function, which carries `model` along with it.

Cell 7 – trace, same seed=3

```
1 _, _, trace = run(model_based_reflex_agent(), seed=3)
2 for step, (loc, action, state) in enumerate(trace, 1):
3     print(f"Step {step}: at {loc} -> action={action:6s} -> world={state}")
```

L1-3

Identical to the previous trace cell, with the agent swapped. Same world, same seed, same harness — so any difference in output is caused by architecture alone. That is the experiment.

PYTHON

Identical to the previous trace cell. Copying a small block like this rather than wrapping it in a function is normal in notebooks, where each cell is meant to be readable on its own.

OUTPUT

```
Step 1: at B -> action=Left    -> world={'A': 'Dirty', 'B': 'Clean'}
Step 2: at A -> action=Suck    -> world={'A': 'Clean', 'B': 'Clean'}
Step 3: at A -> action=NoOp    -> world={'A': 'Clean', 'B': 'Clean'}
Step 4: at A -> action=NoOp    -> world={'A': 'Clean', 'B': 'Clean'}
Step 5: at A -> action=NoOp    -> world={'A': 'Clean', 'B': 'Clean'}
Step 6: at A -> action=NoOp    -> world={'A': 'Clean', 'B': 'Clean'}
```

The first two steps are identical to the reflex agent's. Step 3 is where they part:

STEP	SIMPLE REFLEX	MODEL - BASED	WHY
1	Left	Left	Both see a clean B, both move
2	Suck	Suck	Both see dirt at A
3	Right	NoOp	The model now holds A=Clean <i>and</i> B=Clean, from step 1
4–6	pacing	NoOp	–3 versus 0

THE LESSON

5 → 9 points, from one dictionary. The model-based agent is not smarter about cleaning — it cleans exactly the same dirt on the same step. It is smarter about *stopping*. Memory did not buy better actions, it bought the ability to know the job was finished. Keep that distinction; Exercise 1 is about what happens when that belief goes stale.

07 Architecture 3 — Goal-based

Instead of firing rules, the agent holds an explicit goal — “both squares clean” — and forms a small plan to reach it.

Cell 8 – goal-based agent

```

1  def goal_based_agent():
2      goal_reached_plan = []          # a tiny plan: where to go next once the goal is reached
3
4      def program(percept):
5          location, status = percept
6          nonlocal goal_reached_plan
7          if status == "Dirty":
8              return "Suck"           # always satisfy the immediate subgoal
9          if not goal_reached_plan:
10             move = "Right" if location == "A" else "Left"
11             goal_reached_plan = [move] # plan: go check the other square
12             return goal_reached_plan.pop(0)
13     return program

```

L1

Same factory shape.

PYTHON

Another factory. Every agent in the notebook uses this identical shape, which is what lets the comparison loop treat them interchangeably.

L2

The plan: a list of queued actions. Empty means “no plan right now, make one”. In a real goal-based agent this is where a search algorithm’s output would land; here the world is small enough that the plan is one move long.

PYTHON

An empty list. In an `if`, an empty list counts as **false** and a non-empty one as **true** — that is what line 9’s `if not goal_reached_plan` relies on.

L4–5

The program and the usual unpacking.

PYTHON

The inner function, and tuple unpacking — both exactly as in the previous agents.

L6

`nonlocal` lets line 11 *rebind* `goal_reached_plan` to a new list. Without it, that assignment would create a fresh local variable and the plan would silently vanish between calls. (Line 6 is not needed for the `.pop()` on line 12, which mutates rather than rebinds — it is needed specifically because of line 11.)

PYTHON

`nonlocal` is the subtle one. By default, assigning to a name inside a function creates a *new local variable*, even if a variable of that name exists outside. `nonlocal` says “no, I mean the one in the enclosing function”. Without it, line 11 would build a plan that vanishes the moment the call ends. Note you only need it for *assignment* — calling `.pop()` on line 12 modifies the existing list without rebinding the name, so that would work either way.

L7–8

Dirt under the agent is an immediate sub-goal, satisfied before any planning happens.

PYTHON

Early return, as before.

L9–11

If there is no plan, make one: the only useful thing to do on a clean square is go check the other one.

PYTHON

`not` flips true and false, so this reads “if the plan is empty”.

L12

Pop the front of the plan and execute it. With a one-step plan this is the same as returning the move — but the *shape* is the point: decide a sequence, then follow it.

PYTHON

`.pop(0)` removes the first item from a list *and* returns it, so the list shrinks as the plan is used up.

L13

Return the program, with the plan captured in the closure.

PYTHON

Return the closure.

READ THE CODE, NOT THE LABEL

This agent has a goal in its name but nowhere in its logic. There is no check for “is the goal satisfied?” and therefore **no NoOp branch anywhere**. It always returns `Suck` or a move, so it paces forever exactly like the simple reflex agent — and scores exactly the same, **5**. Watch for this in the comparison table; it is the tutorial's most useful trap. The architecture is only as good as the goal test you actually write.

08 Architecture 4 — Utility-based

A goal is binary: reached or not. A utility function is a number, so it can *trade off* competing concerns — here, cleanliness against a battery budget.

Cell 9 – utility-based agent

```

1 def utility_based_agent(battery_start=6):
2     battery = {"level": battery_start}
3
4     def utility(action, status, battery_level):
5         u = 0
6         if action == "Suck" and status == "Dirty":
7             u += 10
8         if action in ("Left", "Right"):
9             u -= 2 # moving costs energy
10        if battery_level <= 2 and action in ("Left", "Right"):
11            u -= 5 # heavily discourage
12        return u
13
14    def program(percept):
15        location, status = percept
16        if battery["level"] <= 0:
17            return "NoOp" # out of battery – must stop
18        candidates = ["Suck", "Left", "Right", "NoOp"]
19        action = max(candidates, key=lambda a: utility(a, status, battery["level"]))
20        if action in ("Left", "Right", "Suck"):
21            battery["level"] -= 1
22        return action
23    return program

```

L1

The battery budget is a parameter, so you can experiment with scarcity.

PYTHON

A parameter with a default, so `utility_based_agent()` and `utility_based_agent(battery_start=20)` both work.

L2

The battery is wrapped in a dict rather than a bare number for the same reason as before: rebinding a plain `int` inside the nested function would need `nonlocal`, while mutating a dict entry does not.

PYTHON

Why a dict for a single number? Because of the same rule `nonlocal` exists for: `battery = battery - 1` inside the inner function would create a new local variable, but `battery["level"] -= 1` *modifies* the existing dict without rebinding the name. Wrapping a value in a dict is a common way to sidestep the issue.

L4

The utility function. It scores a hypothetical action — nothing is applied to the world. This is the key move of the architecture: the agent can compare options it will not take.

PYTHON

A second inner function. A function can hold as many as it likes, and they can all see the outer function's variables.

L5

Every action starts at zero utility.

PYTHON

Start an accumulator at zero, then adjust it with the rules below.

L6–7

`+10` for sucking actual dirt — deliberately mirroring the environment's own reward on line 24. Note this only fires when the square *is* dirty; `Suck` on a clean square scores 0.

PYTHON

`and` requires both conditions. Note these are separate `if` s, not `elif` s, so several can apply to the same action and their effects stack.

L8–9

`-2` for any move. Note this is *harsher* than the environment's real `-1` — the agent's preferences do not have to match the true performance measure, and the mismatch has consequences.

PYTHON

`in` tests membership: “is `action` one of these?” The parentheses make a tuple; a list would work identically here.

L10–11

A further –5 for moving on a nearly flat battery. This is the actual trade-off: at full charge a move is a minor cost, at low charge it is close to unthinkable.

PYTHON

`<=` is “less than or equal to”. Two conditions joined with `and`, so both must hold.

L12

Return the score.

PYTHON

Hand the number back to whoever called this function.

L14–15

The program proper.

PYTHON

The main program, and the usual unpacking.

L16–17

A hard constraint that overrides the arithmetic: no battery, no action. Some limits should not be negotiable in utility terms.

PYTHON

A **guard clause**: handle the impossible case first and return, so the rest of the function does not need to worry about it.

L18

All four legal actions become candidates — including `NoOp`, which always scores exactly 0 and therefore acts as the **do-nothing baseline**. Any action scoring below 0 is worse than idling.

PYTHON

A plain list of the options to consider.

L19

`max` with a `key` scores every candidate and picks the best. On a tie Python keeps the *first* maximum in list order, so the order on line 18 is a silent tie-breaker — `Suck` wins ties over `NoOp`.

PYTHON

Three things at once. `max(items, key=f)` runs `f` on every item and returns the *item* with the highest result — not the score itself. `lambda a: ...` is a one-line unnamed function, here taking each candidate as `a`. And it reads `status` and `battery` straight from the enclosing scope without them being passed in.

L20–21

Any real action drains one unit of battery; `NoOp` is free.

PYTHON

Membership test again, then subtract one in place.

L22–23

Return the action, then the program.

PYTHON

Return the chosen action; then the outer function returns the program.

WHERE THIS SCORES 10

In the standard six-step run this is the joint best agent. On a clean square, `Suck` and `NoOp` both score 0 while moving scores -2 , so the tie on line 19 goes to `Suck` — a harmless no-op in the environment. It cleans what is under it and never wastes a move.

...AND WHERE IT IS QUIETLY BROKEN

Read that again: **moving is never worth it**. A move always scores -2 , and the utility function has no term for “there might be dirt over there.” This agent only ever scores 10 because it happens to start on the dirty square. Exercise 1 puts it in a world where it has to travel, and it parks on one square forever. The bug is invisible in the tutorial's own test.

09 Architecture 5 — Learning

A learning element that edits the agent's own policy from observed reward — a toy version of the feedback loop behind RLHF.

Cell 10 – learning agent


```

1  def learning_agent():
2      q = {}                                # (state, action) -> learned value
3      last = {"state": None, "action": None}
4
5      def program(percept):
6          state = percept
7          actions = ["Suck", "Left", "Right"]
8          for a in actions:
9              q.setdefault((state, a), 0.0)
10
11         # Learning element: update the value of the last (state, action) using the
12         if last["state"] is not None:
13             reward = 10 if state[1] == "Clean" and last["action"] == "Suck" else
14             key = (last["state"], last["action"])
15             q[key] += 0.5 * (reward - q[key])    # simple running-average update
16
17         # Performance element: mostly exploit the best known action, occasionally
18         if random.random() < 0.1:
19             action = random.choice(actions)
20         else:
21             action = max(actions, key=lambda a: q[(state, a)])
22
23         last["state"], last["action"] = state, action
24         return action
25     return program

```

L1–2

`q` maps a `(state, action)` pair to a learned value — the agent's policy, and the thing that changes over time. This is a miniature Q-table.

PYTHON

`{}` is an empty dict. Its keys will be *tuples* — Python allows that because tuples cannot be changed, and only unchangeable values are allowed as dict keys. (A list could not be used here.)

L3

Remembers the previous step, because the reward for an action only becomes visible on the *next* percept. This one-step delay is the central difficulty of reinforcement learning in miniature.

PYTHON

One dict holding two related values, kept together because they are always updated together.

L5–6

The percept is used directly as the state.

PYTHON

The percept tuple is stored whole this time, without unpacking — the agent uses it as a single identifier for “the situation I am in”.

L7

`NoOp` is deliberately excluded from the action set — the agent is never allowed to learn to idle.

PYTHON

A list of the actions this agent is allowed to consider.

L8–9

`setDefault` lazily initialises unseen pairs to 0.0, so line 21's lookup can never raise.

PYTHON

`.setDefault(key, value)` means “if this key is missing, add it with this value; otherwise leave it alone”. It saves writing an `if key not in q` check. `0.0` with a decimal point is a **float** (a decimal number) rather than a whole-number `int`.

L11–12

The **learning element**, skipped on the very first step when there is no previous action to credit.

PYTHON

`is not None` is the correct way to test against `None` — use `is` / `is not` for `None`, not `==`.

L13

The reward signal: `+10` if the square is now clean *and* the last action was `Suck`, otherwise `-1`. Note what this cannot distinguish — sucking dirt and sucking an

already-clean square both look like success.

PYTHON

A conditional expression again: value-if-true, then the condition, then value-if-false.

`state[1]` takes item number 1 from the tuple — and Python counts from **0**, so index 1 is the *second* item, the status.

L14–15

The update: move the stored value *halfway* toward the observed reward. `0.5` is the learning rate — high, so it adapts within six steps, but it also means the value never settles.

PYTHON

Building a tuple to use as a dict key, then `+=` on that entry. The maths `0.5 * (reward - q[key])` moves the stored value halfway toward the reward.

L17–19

Exploration. 10% of the time, act randomly. Without this the agent would lock onto whatever it tried first and never discover anything better.

PYTHON

`random.random()` returns a decimal between 0 and 1, so `< 0.1` is true roughly 10% of the time — the standard way to write “sometimes”.

L20–21

Exploitation. The other 90%: take the highest-valued action for this state. Together, lines 18–21 are the classic epsilon-greedy policy.

PYTHON

The `max-with-key` pattern again, this time looking each action's stored value up in the dict.

L23–25

Record this step as “last” so the next call can score it, return the action, return the program.

PYTHON

Line 23 assigns two variables on one line: the values on the right are matched up with the targets on the left, in order. Then return the action, and return the program.

WHY IT USES THE GLOBAL RANDOM

Lines 18–19 call `random` directly, not a seeded private generator. That is why the comparison cell has to call `random.seed(0)` before running — without it, this agent's score changes between runs and the table stops being reproducible. It is also why Exercise 1 re-seeds inside its loop.

SIX STEPS IS NOT LEARNING

It scores 10 here, but do not read that as the learning working. With six steps and a learning rate of 0.5 it barely fills its table. What this cell demonstrates is the *shape* of a learning agent — policy, reward, update, explore/exploit — not its competence.

10 Running all five side by side

Same world, same seed, same six steps. Every difference in the table is caused by architecture and nothing else.

Cell 11 – the comparison

```
1 def run(agent_program, steps=6, seed=42):
2     env = VacuumEnvironment(seed=seed)
3     for _ in range(steps):
4         p = env.percept()
5         a = agent_program(p)
6         env.execute(a)
7     return env.performance, env.is_clean(), env.history
8
9
10 random.seed(0) # for reproducibility of the learning agent's exploration
11 agents = [
12     ("Simple Reflex", simple_reflex_agent),
13     ("Model-Based Reflex", model_based_reflex_agent),
14     ("Goal-Based", goal_based_agent),
15     ("Utility-Based", utility_based_agent),
16     ("Learning", learning_agent),
17 ]
18
19 print(f"{'Architecture':22s}{'Performance':>12s}{'All Clean?':>12s}")
20 print("-" * 46)
21 for name, factory in agents:
22     perf, clean, _ = run(factory())
23     print(f"{name:22s}{perf:12d}{str(clean):>12s}")
```

L1–7

`run` defined a second time, character for character identical to cell 4. Harmless — it just makes this cell runnable on its own — but worth noticing so you do not go hunting for a difference that is not there.

PYTHON

Re-running a `def` with the same name simply replaces the old function. No error, no warning — which is worth knowing, because it also means a typo'd redefinition silently overwrites working code.

L10

Seed the *global* generator. This exists solely for the learning agent's exploration on line 18 of the previous cell; without it the last row of the table would wobble.

PYTHON

`random.seed(0)` fixes the starting point of the *global* generator, so any code using `random.something()` produces the same sequence every run.

L11–17

A list of `(name, factory)` pairs. Storing the *factory* rather than a built program is what guarantees each agent starts with fresh internal state.

PYTHON

A list of tuples — a very common Python shape for “a table of rows”. Each row pairs a label with a function. Note the functions are written *without* parentheses, so they are stored rather than called.

L19

The header row. `{'Architecture':22s}` left-aligns in 22 columns, `{'Performance':>12s}` right-aligns in 12 — that is what makes the columns line up.

PYTHON

`{'Architecture':22s}` pads left-aligned to 22 characters; `{'Performance':>12s}` pads right-aligned to 12. The `>` is what flips the alignment.

L20

A rule of 46 dashes, matching $22+12+12$.

PYTHON

`"-" * 46` repeats a string 46 times. Multiplying a string by a number is a genuine Python operation.

L21–22

For each entry, call the factory to build a fresh program and run it. `_` discards the trace.

PYTHON

Looping over a list of tuples unpacks each row into two variables automatically. `factory()` *with* parentheses calls it, producing a fresh agent program.

L23

Print the row. `{perf:12d}` formats the score as a right-aligned integer; `str(clean)` converts the bool before padding it, since `:>12s` requires a string.

PYTHON

`{perf:12d}` formats a whole number right-aligned in 12 columns. `str(clean)` converts True / False into text first, because the `s` format code needs a string.

OUTPUT

Architecture	Performance	All Clean?
Simple Reflex	5	True
Model-Based Reflex	9	True
Goal-Based	5	True
Utility-Based	10	True
Learning	10	True

How to read this table

ARCHITECTURE	SCORE	WHAT ACTUALLY HAPPENED
Simple Reflex	5	Cleans on step 2, then paces for four steps at -1 each.
Model-Based Reflex	9	Same clean, then <code>NoOp</code> — it knows the job is done.
Goal-Based	5	Identical to simple reflex: no goal test, so no stopping.
Utility-Based	10	Starts on the dirty square, sucks, then never pays to move.
Learning	10	Explores into the same behaviour. Six steps is not enough to call this learned.

THE HONEST READING

The scores do **not** rank the architectures. Utility-based ties for first here by being lucky about where it started, goal-based ties for last because of a missing branch, and the learning agent's 10 is barely distinguishable from noise. A six-step, two-square, fully observable, episodic world is simply too easy to separate them.

That is exactly why the exercises exist. Each one breaks a different assumption in the PEAS spec, and each one produces a *different* ranking.

11 Exercise 1 — Dirt that comes back

The task: make dirt reappear at random after being cleaned, turning the world *stochastic* and *non-episodic*. Which architecture degrades least?

My hypothesis before running it: the agents that *stop* once they believe the job is done (model-based, goal-based) should suffer most, because their belief goes stale. The simple reflex agent, which never stops, should suffer least.

Solution – stochastic environment and the fix

```

1  # Exercise 1 – Stochastic, non-episodic dirt
2  #
3  # Dirt can reappear after being cleaned, so the world never "ends".
4  # Hypothesis: the architectures that STOP once they believe the job is done
5  # (Model-Based, Goal-Based) degrade the most, because their belief goes stale.
6  # The Simple Reflex Agent, which never stops patrolling, degrades the least.
7
8  class StochasticVacuumEnvironment(VacuumEnvironment):
9      """Same world, but each Clean square can get dirty again every time step."""
10
11     def __init__(self, dirt_prob=0.5, respawn_prob=0.15, seed=42):
12         super().__init__(dirt_prob=dirt_prob, seed=seed)
13         self.respawn_prob = respawn_prob
14         self.rng = random.Random(seed + 1000) # separate stream, so respawns don't
15         self.steps = 0
16         self.clean_square_steps = 0 # for measuring "how clean was the world"
17
18     def execute(self, action):
19         super().execute(action)
20         for loc in LOCATIONS: # the environment is dirty
21             if self.status[loc] == "Clean" and self.rng.random() < self.respawn_prob:
22                 self.status[loc] = "Dirty"
23         self.steps += 1
24         self.clean_square_steps += sum(1 for v in self.status.values() if v == "Clean")
25         return self.percept()
26
27     def cleanliness(self):
28         """Fraction of (square, time step) pairs that were clean. The right metric
29         for a non-episodic task: there is no final state to score."""
30         return self.clean_square_steps / (self.steps * len(LOCATIONS))
31
32
33 def run_stochastic(agent_program, steps=40, seed=42, respawn_prob=0.15):
34     env = StochasticVacuumEnvironment(seed=seed, respawn_prob=respawn_prob)
35     for _ in range(steps):
36         env.execute(agent_program(env.percept()))
37     return env.performance, env.cleanliness()
38
39
40 STEPS, SEEDS = 40, range(8)
41 stochastic_agents = [
42     ("Simple Reflex", simple_reflex_agent),
43     ("Model-Based Reflex", model_based_reflex_agent),
44     ("Goal-Based", goal_based_agent),

```



```

45     ("Utility-Based",      lambda: utility_based_agent(battery_start=STEPS)), #
46     ("Learning",          learning_agent),
47 ]
48
49 print(f"'Architecture':22s}{'Avg performance':>17s}{'Avg cleanliness':>17s}")
50 print("-" * 56)
51 for name, factory in stochastic_agents:
52     results = []
53     for s in SEEDS:
54         random.seed(0)                # keep the learning agent's exploration
55         results.append(run_stochastic(factory(), steps=STEPS, seed=s))
56     perf = sum(r[0] for r in results) / len(results)
57     clean = sum(r[1] for r in results) / len(results)
58     print(f"{name:22s}{perf:17.1f}{clean:16.0%}")
59
60 print("""
61 Reading the table:
62
63 * Simple Reflex and Goal-Based tie at the top. Neither ever stops: the goal-
64 based agent as written has no NoOp branch, so in this world it degenerates
65 into the same patrol loop as the reflex agent. Constant re-sensing is
66 exactly the right behaviour when dirt keeps coming back.
67 * Model-Based Reflex drops. It latches onto "both squares are Clean", emits
68 NoOp forever, and never looks again - its model is a snapshot of a world
69 that has since changed.
70 * Utility-Based drops furthest. Moving costs -2 while Suck-on-a-clean-square
71 costs 0, so it never pays to walk to the other square. It parks on one
72 square, cleans that one forever, and lets the other rot.
73 * Learning does no better: its reward signal only rewards Suck, so it learns
74 the same park-and-suck policy.
75
76 So the hypothesis was half right. The thing that degrades is not "having a
77 model" - it is *trusting a stale model* (Model-Based) or *pricing movement
78 without pricing neglect* (Utility-Based). The fix for the first is to make
79 beliefs expire: """)
80
81
82 def decaying_model_based_agent(staleness_limit=3):
83     """Model-based, but a belief older than `staleness_limit` steps is not trusted
84     model = {loc: None for loc in LOCATIONS}
85     last_seen = {loc: -10**9 for loc in LOCATIONS}
86     t = {"step": 0}
87
88     def program(percept):

```

```

89         location, status = percept
90         t["step"] += 1
91         model[location] = status
92         last_seen[location] = t["step"]
93         if status == "Dirty":
94             return "Suck"
95         believed_clean = all(
96             model[loc] == "Clean" and t["step"] - last_seen[loc] <= staleness_li
97             for loc in LOCATIONS
98         )
99         if believed_clean:
100             return "NoOp"
101         return "Right" if location == "A" else "Left"
102     return program
103
104
105 results = [run_stochastic(decaying_model_based_agent(), steps=STEPS, seed=s) for
106 print(f"\n{'Model-Based + decay':22s}"
107       f"{sum(r[0] for r in results)/len(results):17.1f}"
108       f"{sum(r[1] for r in results)/len(results):16.0%}")

```

L1–6

The hypothesis, written down before the code runs. Worth doing: half of it turns out to be wrong, and that is the interesting part.

PYTHON

Six comment lines. Python ignores everything after a `#`, so comments cost nothing to run — they exist purely for whoever reads the code next, including you in a week.

L8–9

Subclass rather than edit the original, so every earlier cell still works and the two worlds can be compared directly.

PYTHON

`class Child(Parent)` is **inheritance**: the new class automatically gets every method of the old one, and you override only what should differ. Everything not mentioned here still works exactly as before.

L11–12

`super().__init__` builds the ordinary starting world, so the only difference is what happens afterwards.

PYTHON

`super()` means “the parent class”, so `super().__init__(...)` runs the original constructor before adding anything new.

L13

How likely a clean square is to become dirty again, each step.

PYTHON

Storing a constructor argument as an attribute so the other methods can reach it.

L14

A *separate* random stream, offset by 1000. Without this, respawn draws would consume numbers from the same generator that built the starting world, and changing `respawn_prob` would silently change the starting layout too — you could no longer tell which variable caused a difference.

PYTHON

`seed + 1000` is ordinary arithmetic producing a different seed, and therefore a different sequence of random numbers.

L15–16

Counters for the new metric.

PYTHON

Two counters starting at zero.

L18–19

Override `execute`; delegate the action's normal effects to the parent.

PYTHON

Defining a method the parent already has **overrides** it. Calling `super().execute(action)` first runs the parent's version, then this method adds behaviour on top — extending rather than replacing.

L20–22

The change that matters. After the agent acts, the environment acts on its own: each clean square may become dirty again. The world is no longer something that only changes when the agent touches it.

PYTHON

A `for` loop over the location names, with an `if` inside it. `and` requires both conditions to hold.

L23–24

Accumulate how many squares were clean at each step.

PYTHON

`sum(...)` adds up the values a generator expression produces; `1 for v in ... if v == "Clean"` yields a 1 for each clean square, so the sum counts them.

L25

Return the fresh percept, matching the parent's contract.

PYTHON

Returning a value from the overriding method, matching what the parent returned.

L27–30

A new performance measure. `is_clean()` is meaningless now — there is no final state, and whether the world happens to be clean on the last step is luck. Instead: what fraction of (square, time step) pairs were clean? Changing the environment forced changing the metric, which is the deeper lesson of this exercise.

PYTHON

A docstring spanning two lines. `/` is division and `len(...)` gives the number of items in a collection.

L33–37

A runner for the new world, structured like the original but returning the new metric.

PYTHON

Several parameters with defaults. Line 36 nests calls: `env.percept()` runs first, its result goes into `agent_program(...)`, and that result goes into `env.execute(...)` — innermost first, always.

L40

40 steps and 8 seeds — long enough for respawns to matter, repeated enough to average out lucky worlds.

PYTHON

Assigning two variables on one line. `range(8)` is stored for reuse rather than looped over immediately.

L41–47

The same five architectures. Line 45 wraps the utility agent in a `lambda` to give it a battery of 40 instead of 6 — otherwise it would simply run flat at step 6 and the comparison would measure battery size, not architecture.

PYTHON

Line 45 wraps a call in `lambda:` with no arguments. Writing `utility_based_agent(battery_start=STEPS)` directly would call it *now*; wrapping it in a `lambda` stores it to be called later, matching the other rows which hold uncalled functions.

L49–50

Header and rule, same formatting trick as the original comparison.

PYTHON

Header formatting, same technique as the earlier comparison cell.

L51–55

For each agent, run every seed. `random.seed(0)` inside the loop keeps the learning agent reproducible.

PYTHON

A loop inside a loop. The inner one runs every seed; `results.append(...)` collects each outcome into the list created on line 52.

L56–58

Average both metrics and print the row. `{clean:16.0%}` formats a fraction as a whole-number percentage.

PYTHON

`r[0]` takes the first item of each tuple. `sum(...)/len(...)` is how you average a list in plain Python. `{clean:16.0%}` multiplies by 100 and appends a percent sign, with `.0` meaning no decimal places.

L60–79

The written analysis, printed with the results so the conclusion never drifts from the numbers it describes.

PYTHON

One call to `print` with a single triple-quoted string spanning twenty lines. Triple quotes (`"""`) let a string contain line breaks; everything between them is printed exactly as typed.

L82–83

The fix. A model-based agent whose beliefs expire.

PYTHON

Backticks inside a docstring are just characters — Python has no special meaning for them; that is a convention borrowed from Markdown.

L84–86

Same model as before, plus a record of *when* each square was last observed, and a step counter. Line 85's `-10**9` is a sentinel meaning “never seen”, guaranteed to look infinitely stale.

PYTHON

`-10**9` is negative ten to the ninth power. `**` is exponentiation, not multiplication.

L88–92

Advance the clock, record the observation and its timestamp.

PYTHON

Reading a dict entry, adding to it, and writing it back.

L93–94

Unchanged reflex rule.

PYTHON

Early return.

L95–98

The key condition. The agent now believes the world is clean only if every square is *both* believed clean *and* was observed within the last `staleness_limit` steps. Knowledge has an expiry date.

PYTHON

`all(...)` with a generator expression spread over several lines — legal because the open bracket on line 95 is not closed until line 98, and Python ignores line breaks inside brackets.

L99–101

Idle if that holds; otherwise patrol.

PYTHON

A boolean variable used directly as a condition. Writing `if believed_clean` is preferred over `if believed_clean == True`.

L105–108

Run the fixed agent on the same seeds and print its row under the table for comparison.

PYTHON

Line 105 is a **list comprehension**: `[expression for item in collection]` builds a list in one line. Lines 106–108 are three f-strings written next to each other with no operator between them — Python automatically joins adjacent string literals into one. `\n` is an escape sequence meaning a newline character.

OUTPUT

Architecture	Avg performance	Avg cleanliness

Simple Reflex	93.4	73%
Model-Based Reflex	67.8	52%
Goal-Based	93.4	73%
Utility-Based	58.8	45%
Learning	58.9	45%

Reading the table:

- * Simple Reflex and Goal-Based tie at the top. Neither ever stops: the goal-based agent as written has no NoOp branch, so in this world it degenerates into the same patrol loop as the reflex agent. Constant re-sensing is exactly the right behaviour when dirt keeps coming back.
- * Model-Based Reflex drops. It latches onto "both squares are Clean", emits NoOp forever, and never looks again - its model is a snapshot of a world that has since changed.
- * Utility-Based drops furthest. Moving costs -2 while Suck-on-a-clean-square costs 0, so it never pays to walk to the other square. It parks on one square, cleans that one forever, and lets the other rot.
- * Learning does no better: its reward signal only rewards Suck, so it learns the same park-and-suck policy.

So the hypothesis was half right. The thing that degrades is not "having a model" - it is **trusting a stale model** (Model-Based) or **pricing movement without pricing neglect** (Utility-Based). The fix for the first is to make beliefs expire:

Model-Based + decay	100.1	69%
---------------------	-------	-----

What the numbers actually said

The hypothesis was **half right**, and being wrong about the other half is the most useful thing in this exercise.

ARCHITECTURE	PERF.	CLEAN	WHAT HAPPENED
Simple Reflex	93.4	73%	Never stops, so it re-senses constantly. Memorylessness is now an <i>advantage</i> .
Goal-Based	93.4	73%	Did not collapse. It has no <code>NoOp</code> branch, so it degenerates into the same patrol loop.
Model-Based	67.8	52%	Latches onto “both clean” and idles forever on a snapshot of a world that has moved on.
Utility-Based	58.8	45%	Worst. Moving scores -2, sucking a clean square scores 0 — so it parks on one square and lets the other rot.
Learning	58.9	45%	Its reward only credits <code>Suck</code> , so it learns the same park-and-suck policy.
Model-Based + decay	100.1	69%	Best performance of all: patrols like the reflex agent but skips pointless moves.

THE REAL ANSWER

What degrades is not *having a model*. It is **trusting a stale model** (model-based) or **pricing movement without pricing neglect** (utility-based). Note the fix does not remove the memory — it adds an expiry date to it, and comes out ahead of every original agent.

Also note the utility agent's bug was invisible in the original tutorial. Its 10/10 score there came from starting on the dirty square. This exercise is what exposed it.

12 Exercise 2 — Taking away the location sensor

The task: `percept()` returns only the dirt status, never the location. Which architectures still work, which break, and why?

Solution – three agents under partial observability


```

1  # Exercise 2 – Partial observability: the agent can no longer see WHERE it is
2  #
3  # percept() now returns only the dirt status of the current square.
4  # Question: which architectures survive?
5
6  class BlindVacuumEnvironment(VacuumEnvironment):
7      """Sensors are degraded: dirt status only, no location."""
8
9      def percept(self):
10         return self.status[self.agent_location]      # a bare "Dirty"/"Clean", no
11
12
13 def run_blind(agent_program, steps=6, seed=3):
14     env = BlindVacuumEnvironment(seed=seed)
15     for _ in range(steps):
16         env.execute(agent_program(env.percept()))
17     return env.performance, env.is_clean(), env.history
18
19
20 # --- (a) Simple reflex with a fixed rule: BREAKS -----
21 def blind_fixed_reflex_agent():
22     """No memory, no location. The best it can do is one fixed move direction."""
23     def program(status):
24         return "Suck" if status == "Dirty" else "Right"
25     return program
26
27
28 # --- (b) Simple reflex + randomisation: WORKS, eventually -----
29 def blind_random_reflex_agent(seed=7):
30     """AIMA's answer to unobservable location: randomise the move.
31     Still memoryless, but no longer trapped in a corner."""
32     rng = random.Random(seed)
33     def program(status):
34         return "Suck" if status == "Dirty" else rng.choice(["Left", "Right"])
35     return program
36
37
38 # --- (c) Model-based: WORKS, and works perfectly -----
39 def blind_model_based_agent():
40     """Key insight: the ACTUATORS are absolute and deterministic - "Left" always
41     lands on A, "Right" always lands on B. So after the agent's first move it
42     knows exactly where it is, even though the sensors never tell it.
43     Internal state has *reconstructed* the observability the sensors lost."""
44     belief = {"location": None, "A": None, "B": None}

```

```

45
46     def program(status):
47         loc = belief["location"]
48         if loc is not None:
49             belief[loc] = status                # we know where this read
50         if status == "Dirty":
51             if loc is not None:
52                 belief[loc] = "Clean"          # Suck will succeed
53             return "Suck"
54         if loc is None:                          # first clean percept: loc
55             belief["location"] = "A"
56             return "Left"                       # "Left" is a teleport-to
57         if belief["A"] == "Clean" and belief["B"] == "Clean":
58             return "NoOp"
59         belief["location"] = "B" if loc == "A" else "A"
60         return "Right" if loc == "A" else "Left"
61     return program
62
63
64 for label, factory in [("a) fixed reflex ", blind_fixed_reflex_agent),
65                       ("b) random reflex ", blind_random_reflex_agent),
66                       ("c) model-based ", blind_model_based_agent)]:
67     perf, clean, trace = run_blind(factory())
68     actions = " ".join(a for _, a, _ in trace)
69     print(f"{label} perf={perf:4d} all clean={str(clean):5s} actions: {actions}")
70
71 print("""
72 * (a) Fixed reflex BREAKS. With no location in the percept it cannot choose a
73 direction, so it commits to one - "Right" - forever. In this run it starts
74 on a clean B and simply re-enters B six times, paying the move penalty each
75 step while the dirt in A is never found (perf -6, world never clean). Note
76 it does not fail loudly; it just quietly stops making progress.
77 * (b) Randomised reflex WORKS, in expectation. Still no memory, but a coin flip
78 guarantees it visits both squares eventually. This is the classic result:
79 when you cannot sense, randomise.
80 * (c) Model-based WORKS, and cleanly. Because Left/Right are absolute moves,
81 one deliberate "Left" tells the agent it is now at A - it has localised
82 itself using its own action history rather than its sensors. From there it
83 tracks position by dead reckoning and can still emit NoOp when done.
84
85 The general rule: partial observability breaks memoryless agents, and internal
86 state is what buys the observability back. That is precisely why the
87 model-based box exists in the AIMA hierarchy.""")

```

L1–4

The setup: this is a *sensor* downgrade, breaking the S in PEAS.

PYTHON

Comment lines setting up what this cell does. The `#` makes Python skip the rest of the line entirely.

L6–10

Subclass overriding one method. `percept` now returns a bare `"Dirty" / "Clean"` string instead of a tuple. That is the entire change — one line, and it invalidates most of the agents written so far.

PYTHON

A subclass overriding a single method. Everything else — the constructor, `execute`, `is_clean` — is inherited untouched.

L13–17

A runner for the blind world, otherwise identical to the original.

PYTHON

The same runner shape as before, with a different environment class inside.

L21–25

(a) The memoryless agent. Its program now takes `status` directly, since there is no tuple to unpack. With no location it cannot choose a direction, so line 24 hard-codes one: suck dirt, otherwise always go right.

PYTHON

Note the parameter is now `status`, not `percept`: the inner function's parameter name is entirely up to you, and here it is renamed to reflect that a bare string arrives rather than a tuple. Line 24 returns a conditional expression directly.

L29–35

(b) Randomised. Identical, except line 34 flips a coin instead of committing to a direction. Still completely memoryless — the `rng` is not state about the world, just a source of variety.

PYTHON

The docstring spans two lines. Line 32 creates a private generator so this agent's coin flips do not disturb anyone else's random numbers.

L39–43

(c) Model-based — the interesting one. The docstring holds the insight: the *actuators* are absolute and deterministic. `Left` always lands on A regardless of where you were. So the agent can learn its position from its own *actions*, even though its sensors will never tell it.

PYTHON

A four-line docstring. The `*` characters inside it are just text.

L44

Belief state: where I think I am, plus what I have seen at each square.

PYTHON

One dict holding three different things — a quick alternative to defining a small class.

L46–49

If the agent knows where it is, it can attribute the reading to that square. If it does not, the reading is unattributable and must be discarded.

PYTHON

Reading a dict value into a local variable, then an `is not None` test. Line 49 uses a variable as the dict key, which is why the belief can be recorded against whichever square the agent thinks it is on.

L50–53

Dirt gets sucked regardless of whether the agent knows where it is — that rule never needed a location. Line 52 pre-records the result, since `Suck` is guaranteed to succeed on a dirty square.

PYTHON

An `if` nested inside another `if` — the deeper indentation is what marks the nesting.

L54–56

The self-localisation move. Lost, on a clean square: deliberately go `Left`. Because that is a teleport-to-A, the agent *now knows* it is at A. It has spent one move to convert “I am lost” into “I am at A”.

PYTHON

`is None` tests for the “nothing yet” value.

L57–58

Once both squares are known clean, idle — a capability the blind reflex agents cannot express at all.

PYTHON

Two equality tests joined with `and`.

L59–60

Otherwise move to the other square, updating the belief about position *before* moving. This is dead reckoning: tracking position by remembering your own actions.

PYTHON

Two conditional expressions in a row: one assigns, the next returns.

L64–69

Run all three and print each one's full action sequence, which is what makes the failure visible.

PYTHON

Looping directly over a list of tuples written inline in the `for` statement. Line 68 is a **join**: `" ".join(...)` glues a sequence of strings together with a space between each. The `a`

```
for _, a, _ in trace inside it pulls out only the middle item of each triple, discarding
the other two with _ .
```

L71–87

The analysis, printed alongside.

PYTHON

A single `print` of one long triple-quoted string. The `*` at the start of several lines is just a character — Python attaches no meaning to it inside a string.

OUTPUT

```
(a) fixed reflex   perf=  -6  all clean=False  actions: Right Right Right Right Right Rig
(b) random reflex  perf=   5  all clean=True   actions: Right Left Suck Right Left Left
(c) model-based    perf=   8  all clean=True   actions: Left Suck Right NoOp NoOp NoOp
```

- * (a) Fixed reflex BREAKS. With no location in the percept it cannot choose a direction, so it commits to one - "Right" - forever. In this run it starts on a clean B and simply re-enters B six times, paying the move penalty each step while the dirt in A is never found (perf -6, world never clean). Note it does not fail loudly; it just quietly stops making progress.
- * (b) Randomised reflex WORKS, in expectation. Still no memory, but a coin flip guarantees it visits both squares eventually. This is the classic result: when you cannot sense, randomise.
- * (c) Model-based WORKS, and cleanly. Because Left/Right are absolute moves, one deliberate "Left" tells the agent it is now at A - it has localised itself using its own action history rather than its sensors. From there it tracks position by dead reckoning and can still emit NoOp when done.

The general rule: partial observability breaks memoryless agents, and internal state is what buys the observability back. That is precisely why the model-based box exists in the AIMA hierarchy.

The verdict

AGENT	RESULT	WHY
(a) Fixed reflex	Breaks (−6, never clean)	Committed to <code>Right</code> , started on a clean B, and re-entered B six times. The dirt in A was never found.
(b) Random reflex	Works, eventually (+5)	A coin flip guarantees it visits both squares given enough steps. When you cannot sense, randomise.
(c) Model-based	Works perfectly (+8)	<code>Left</code> → <code>Suck</code> → <code>Right</code> → <code>NoOp</code> . It localised itself, cleaned, verified, and stopped.

THE MOST IMPORTANT DETAIL

Agent (a) does not crash. It raises nothing, logs nothing, and returns a legal action every single step. It just quietly stops making progress while paying a penalty. Degraded sensors produce *silent* failures, which is precisely why you cannot catch this class of bug by checking that the code runs.

THE GENERAL RULE

Partial observability breaks memoryless agents, and internal state is what buys the observability back. Agent (c) never gains a location sensor — it *reconstructs* location from its own action history. That is the whole reason the model-based box exists in the hierarchy.

13 Exercise 3 — Scaling to a row of four

The task: extend `LOCATIONS` to `["A", "B", "C", "D"]` in a row, update the movement logic, and ask whether the simple reflex agent still reaches `NoOp` sensibly.

SCALING FORCES A SEMANTIC CHANGE

With two squares, `Left` could be *absolute* — “go left” and “go to A” were the same instruction. In a row of four they are not. Movement has to become *relative*, and that single change is what makes this exercise more than bookkeeping. (It is also what Exercise 2’s agent (c) was quietly exploiting.)

```

1  # Exercise 3 – Scale up to a row of four squares: A - B - C - D
2  #
3  # Note the change of semantics: with two squares "Left"/"Right" could be
4  # absolute (teleport to A / teleport to B). In a row of four they must become
5  # RELATIVE moves, which is what makes the exercise interesting.
6
7  ROW = ["A", "B", "C", "D"]
8
9
10 class RowVacuumEnvironment:
11     """Same PEAS, but n squares in a row and Left/Right are one step, clamped at
12
13     def __init__(self, locations=ROW, dirt_prob=0.5, seed=42):
14         rng = random.Random(seed)
15         self.locations = list(locations)
16         self.status = {loc: ("Dirty" if rng.random() < dirt_prob else "Clean") for loc in self.locations}
17         self.index = rng.randrange(len(self.locations))
18         self.performance = 0
19         self.history = []
20
21     @property
22     def agent_location(self):
23         return self.locations[self.index]
24
25     def percept(self):
26         return (self.agent_location, self.status[self.agent_location])
27
28     def execute(self, action):
29         loc = self.agent_location
30         if action == "Suck":
31             if self.status[loc] == "Dirty":
32                 self.status[loc] = "Clean"
33                 self.performance += 10
34             elif action == "Left":
35                 self.index = max(0, self.index - 1)          # clamp: bumping into t
36                 self.performance -= 1
37             elif action == "Right":
38                 self.index = min(len(self.locations) - 1, self.index + 1)
39                 self.performance -= 1
40             elif action == "NoOp":
41                 pass
42             else:
43                 raise ValueError(f"Unknown action: {action}")
44             self.history.append((loc, action, dict(self.status)))

```

```

45         return self.percept()
46
47     def is_clean(self):
48         return all(v == "Clean" for v in self.status.values())
49
50
51 def run_row(agent_program, steps=14, seed=42):
52     env = RowVacuumEnvironment(seed=seed)
53     for _ in range(steps):
54         env.execute(agent_program(env.percept()))
55     return env.performance, env.is_clean(), env.history
56
57
58 def row_simple_reflex_agent(locations=ROW):
59     """Memoryless. The only thing it can key off is (location, status), so the
60     best fixed rule is: sweep right, and at the right wall sweep left."""
61     def program(percept):
62         location, status = percept
63         if status == "Dirty":
64             return "Suck"
65         return "Left" if location == locations[-1] else "Right"
66     return program
67
68
69 def row_model_based_agent(locations=ROW):
70     """Internal state: what we have seen at every square. Sweep right to the wall
71     then left, and stop once all four are believed clean."""
72     model = {loc: None for loc in locations}
73     direction = {"d": "Right"}
74
75     def program(percept):
76         location, status = percept
77         model[location] = status
78         if status == "Dirty":
79             model[location] = "Clean"
80             return "Suck"
81         if all(v == "Clean" for v in model.values()):
82             return "NoOp"
83             # every square seen,
84         if location == locations[-1]:
85             direction["d"] = "Left"
86         elif location == locations[0]:
87             direction["d"] = "Right"
88         return direction["d"]
89     return program

```



```

89
90
91 for label, factory in [("Simple Reflex", row_simple_reflex_agent),
92                        ("Model-Based Reflex", row_model_based_agent)]:
93     perf, clean, trace = run_row(factory())
94     actions = " ".join(a for _, a, _ in trace)
95     print(f"{label} perf={perf:4d} all clean={str(clean):5s}")
96     print(f"'':20s}actions: {actions}")
97
98 print("""
99 Does the Simple Reflex Agent still reach NoOp sensibly? No - and it is worse
100 than that. Look at its action trace: after cleaning its way right it settles
101 into "Left Right Left Right ..." - it is not patrolling the row, it is stuck
102 oscillating between the last two squares.
103
104 Why: the rule "Left at the right wall, otherwise Right" makes the right wall a
105 2-cycle. Direction of travel is *state*, and a memoryless agent has nowhere to
106 keep it, so it cannot commit to a leftward sweep. With n=4 it gets away with
107 this, because it cleaned A-D on the way out. With n=8 it does not - it strands
108 itself at the far end while dirt sits behind it: """)
109
110 for n in (2, 4, 8):
111     locations = [chr(ord("A") + i) for i in range(n)]
112     for label, factory in [("Simple Reflex", row_simple_reflex_agent),
113                           ("Model-Based ", row_model_based_agent)]:
114         env = RowVacuumEnvironment(locations=locations, seed=42)
115         program = factory(locations)
116         for _ in range(4 * n):
117             env.execute(program(env.percept()))
118         print(f" n={n} {label} perf={env.performance:4d} clean={env.is_clean}")
119
120 print("""
121 And NoOp is out of reach for a second, independent reason: NoOp is only
122 rational once you know *every* square is clean, and "every square" is a fact
123 about squares you are not standing on. A memoryless agent has no place to put
124 that fact, so its rule set cannot even express the NoOp condition. The
125 model-based agent keeps both pieces of state - direction of travel and what it
126 has seen where - so it sweeps properly and then stops.
127
128 Exercise 2's lesson applies here too: if you must stay memoryless, randomise.""")
129
130
131 def row_random_reflex_agent(locations=ROW, seed=7):
132     """Memoryless, but breaks the oscillation trap with a coin flip."""

```

```

133     rng = random.Random(seed)
134     def program(percept):
135         _, status = percept
136         return "Suck" if status == "Dirty" else rng.choice(["Left", "Right"])
137     return program
138
139
140 for n in (4, 8):
141     locations = [chr(ord("A") + i) for i in range(n)]
142     cleaned, perfs = 0, []
143     for trial in range(20):
144         env = RowVacuumEnvironment(locations=locations, seed=42)
145         program = row_random_reflex_agent(locations, seed=trial)
146         for _ in range(4 * n):
147             env.execute(program(env.percept()))
148             cleaned += env.is_clean()
149             perfs.append(env.performance)
150     print(f"  n={n}  Random Reflex over {4*n} steps: cleaned the row in "
151           f"{cleaned}/20 trials, avg perf {sum(perfs)/len(perfs):.1f}")
152
153 print("""
154 Randomising escapes the trap but is slow: a random walk's cover time grows
155 like n-squared, so within a fixed step budget it often has not visited
156 everything yet. Memorylessness is survivable, not free - which is the point of
157 the exercise.""")

```

L1–7

The setup and the new world: four squares in a row.

PYTHON

Lines 1–5 are comments — everything after `#` is ignored by Python, so a block like this is free to write. Line 7 creates a new module-level list; nothing stops you having several.

L10–11

A fresh class rather than a subclass — the movement model differs too much to inherit cleanly.

PYTHON

`class RowVacuumEnvironment:` with no parentheses means it inherits nothing — a fresh start rather than a subclass.

L13–16

As before, but the world size is now a parameter. Line 16's comprehension is unchanged except that it iterates over `self.locations`.

PYTHON

`locations=ROW` gives a list as a default value. Line 15's `list(locations)` makes a copy, which matters: without it, the instance and the caller would share one list, and changing either would change both.

L17

Position is now an **index**, not a name. That is the core representational change: in a row you need to know how far along you are, not just which label you are on.

PYTHON

`rng.randrange(n)` returns a whole number from 0 up to $n-1$ — exactly the valid index range for a list of length n .

L18–19

Scorecard and trace, unchanged.

PYTHON

Two more attributes initialised to empty starting values, exactly as in the original class.

L21–23

A `@property` so `agent_location` still reads like an attribute, keeping every agent's `percept` unpacking working unchanged. The old interface is preserved over a new representation.

PYTHON

`@property` is a **decorator** — a line starting with `@` that modifies the function below it. This one lets you write `env.agent_location` with no parentheses, even though it is really a method being run. It is how you replace a stored attribute with a computed one without changing any code that reads it.

L25–26

Identical percept contract: current location and its status.

PYTHON

Note it calls the property by name, no parentheses — that is the point of line 21.

L28–33

`Suck` is unchanged — it never depended on the world's shape.

PYTHON

Unchanged from the original class. Repeating code rather than inheriting is a deliberate trade here: more duplication, but each class reads on its own.

L34–36

Left is now relative: one step back, `max(0, ...)` clamping at the wall. Note the `-1` is charged *even when the move is blocked* — walking into a wall costs you.

PYTHON

`max(0, self.index - 1)` takes the larger of the two, so the result can never drop below 0. It is the idiomatic Python way to clamp a lower bound — no `if` needed.

L37–39

`Right` mirrors it, clamped at the far end.

PYTHON

`min(...)` is the mirror image, clamping the upper bound at the last valid index.

L40–43

`NoOp` and the error branch, unchanged.

PYTHON

The same `pass` and `raise` pattern as the original.

L44–45

`dict(self.status)` is a shallow copy — sufficient here, because the values are immutable strings.

PYTHON

`dict(self.status)` makes a copy of a dict — a *shallow* one, meaning it copies the entries but not the values inside them. Here the values are plain strings, which cannot change, so shallow is enough and `deepcopy` would be wasted work.

L47–55

`is_clean` and the runner, unchanged in structure.

PYTHON

Familiar shapes: `all()` with a generator, then a loop calling the agent.

L58–66

The reflex agent, scaled. Memoryless, so its rule can only key off `(location, status)`. Line 65 is the best fixed rule available: sweep right, and reverse at the right wall.

PYTHON

`locations[-1]` is **negative indexing**: `-1` means the last item, `-2` the second-to-last. Much cleaner than `locations[len(locations) - 1]`.

L69–73

The model-based agent, scaled. Two pieces of state now: what it has seen at every square (line 72) *and* which way it is travelling (line 73). That second one is new — it did not need it with two squares.

PYTHON

Two pieces of state in the closure this time. Line 72 builds a dict from a list with a comprehension.

L75–80

Observe, then suck dirt. Line 79 pre-records the success.

PYTHON

The usual observe-then-act shape.

L81–82

Idle once every square is known clean. `all(...)` over the model is `False` while any entry is still `None`, so the agent cannot stop before visiting everywhere — the unvisited state is doing real work.

PYTHON

`all()` over `.values()`. Note that `None` is not equal to `"Clean"`, so an unvisited square makes this false automatically — no extra check needed.

L83–87

Reverse direction at either wall, then travel that way. This is the sweep the reflex agent cannot perform.

PYTHON

`locations[0]` is the first item. Assigning into the `direction` dict changes state that persists to the next call.

L91–96

Run both and print their action sequences.

PYTHON

Line 95 mixes formats: `{perf:4d}` pads a number to 4 columns, `{str(clean):5s}` converts then pads. Line 96's `{' ':20s}` prints 20 spaces — a quick way to indent under the previous line.

L98–108

The analysis of what those sequences show.

PYTHON

A multi-line string passed straight to `print`. Everything inside the triple quotes appears exactly as written, line breaks included.

L110–118 **The scaling test.** Same agents at $n=2, 4$ and 8 , with a step budget of `4n` so the budget scales with the problem. Line 111 generates location names with `chr(ord('A') + i)`.

PYTHON

`ord("A")` gives the numeric code for the letter A, and `chr(...)` turns a number back into a letter — so `chr(ord("A") + i)` generates A, B, C, D... Line 116 multiplies inside `range`, which is fine since it is just arithmetic.

L120–128 The second half of the analysis: the independent reason `NoOp` is unreachable.

PYTHON

Another multi-line string.

L131–137 A randomised memoryless agent, applying Exercise 2's lesson to this problem.

PYTHON

Line 135 unpacks a tuple but keeps only the second part, discarding the first with `_`.

L140–151 20 trials per size, since a random agent's single run tells you nothing. Line 148 exploits `True == 1` to count successes.

PYTHON

Line 142 assigns two variables at once, one a number and one a list. Line 148 uses `cleaned += env.is_clean()` — in Python `True` equals 1 and `False` equals 0, so adding booleans counts how many were true. Line 150–151 splits one f-string across two lines by placing them next to each other inside the call's parentheses.

L153–157 The closing caveat about how slow random search actually is.

PYTHON

A final multi-line string.

OUTPUT

```
Simple Reflex      perf= 19  all clean=True
                   actions: Right Suck Right Suck Right Suck Left Right Left Right Left
Model-Based Reflex perf= 27  all clean=True
                   actions: Right Suck Right Suck Right Suck NoOp NoOp NoOp NoOp NoOp No
```

Does the Simple Reflex Agent still reach NoOp sensibly? No - and it is worse than that. Look at its action trace: after cleaning its way right it settles into "Left Right Left Right ..." - it is not patrolling the row, it is stuck oscillating between the last two squares.

Why: the rule "Left at the right wall, otherwise Right" makes the right wall a 2-cycle. Direction of travel is **state**, and a memoryless agent has nowhere to keep it, so it cannot commit to a leftward sweep. With $n=4$ it gets away with this, because it cleaned A-D on the way out. With $n=8$ it does not - it strands itself at the far end while dirt sits behind it:

```
n=2 Simple Reflex perf= 3  clean=True
n=2 Model-Based   perf= 9  clean=True
n=4 Simple Reflex perf= 17 clean=True
n=4 Model-Based   perf= 27 clean=True
n=8 Simple Reflex perf= -21 clean=False
n=8 Model-Based   perf= 32 clean=True
```

And NoOp is out of reach for a second, independent reason: NoOp is only rational once you know **every** square is clean, and "every square" is a fact about squares you are not standing on. A memoryless agent has no place to put that fact, so its rule set cannot even express the NoOp condition. The model-based agent keeps both pieces of state - direction of travel and what it has seen where - so it sweeps properly and then stops.

Exercise 2's lesson applies here too: if you must stay memoryless, randomise.

```
n=4 Random Reflex over 16 steps: cleaned the row in 14/20 trials, avg perf 13.7
n=8 Random Reflex over 32 steps: cleaned the row in 13/20 trials, avg perf 4.8
```

Randomising escapes the trap but is slow: a random walk's cover time grows like n -squared, so within a fixed step budget it often has not visited everything yet. Memorylessness is survivable, not free - which is the point of the exercise.

The answer: no, and worse than you would expect

Look at the reflex agent's trace: after cleaning its way right it settles into Left Right Left Right It is not patrolling the row — it is **stuck oscillating between the last two squares**.

The rule “Left at the right wall, otherwise Right” makes that wall a 2-cycle: step left off the wall, and the rule immediately sends you back. **Direction of travel is state**, and a memoryless agent has nowhere to keep it, so it cannot commit to a leftward sweep. At $n=4$ it gets away with this because it cleaned everything on the way out. At $n=8$ it does not:

ROW SIZE	SIMPLE REFLEX	MODEL-BASED	RANDOM REFLEX (20 TRIALS)
$n = 2$	3, clean	9, clean	—
$n = 4$	17, clean	27, clean	cleaned 14/20, avg 13.7
$n = 8$	-21, never clean	32, clean	cleaned 13/20, avg 4.8

And NoOp is out of reach for a second, independent reason: it is only rational once you know every square is clean — a fact about squares you are not standing on. A memoryless agent has no place to put that fact, so its rule set cannot even express the stopping condition.

RANDOMISING IS SURVIVABLE, NOT FREE

The coin flip escapes the oscillation trap, but a random walk's cover time grows like n^2 , so inside a fixed step budget it frequently has not visited everything yet — it fails to clean the row in about a third of trials at both sizes. Memory is not a luxury here; it is the difference between $O(n)$ and $O(n^2)$.

14 Exercise 4 — Adding time pressure

The task: add a time-pressure term to `utility_based_agent` — a deadline after which `Suck` is worth less — and observe how the agent's choices shift.

Two supporting repairs are needed first, both diagnosed in Exercise 1. The original utility agent prices a move at a flat -2 with no term for what it might find, so **travel is never worth it**; and its beliefs never expire, so a square it cleaned long ago still looks clean. Adding a deadline to an agent that refuses to move would demonstrate nothing.

Solution — deadline, lookahead and decaying belief


```

1  # Exercise 4 – Add time pressure to the utility function
2  #
3  # Three changes to `utility_based_agent`:
4  #   1. A DEADLINE term: a clean square is worth full value up to the deadline
5  #       and decays afterwards ("the shift is ending, cleaning matters less").
6  #   2. A one-step LOOKAHEAD, so moving can be worth something. In the original,
7  #       moving scored a flat -2 while Suck-on-a-clean-square scored 0, so the
8  #       agent could never justify walking anywhere - the parking bug Exercise 1
9  #       exposed.
10 #   3. A DECAYING BELIEF, so a square we cleaned long ago looks suspicious
11 #       again - Exercise 1's staleness fix, expressed as a probability instead
12 #       of a hard cutoff.
13 #
14 # Runs in Exercise 1's StochasticVacuumEnvironment, so run that cell first.
15
16 def utility_agent_with_deadline(battery_start=6, deadline=4, decay=0.6,
17                                 respawn_belief=0.15, verbose=False):
18     state = {"battery": battery_start, "t": 0}
19     belief = {loc: "Unknown" for loc in LOCATIONS}
20     last_seen = {loc: 0 for loc in LOCATIONS}
21
22     def time_factor(t):
23         """What a clean square is worth at time t: full value up to the deadline
24         return 1.0 if t <= deadline else decay ** (t - deadline)
25
26     def p_dirty(dest, t):
27         """How likely is the square over there to be dirty, given what we last s
28         if belief[dest] == "Dirty":
29             return 1.0
30         if belief[dest] == "Unknown":
31             return 0.5
32         age = t - last_seen[dest] # the longer ago w
33         return 1 - (1 - respawn_belief) ** age
34
35     def utility(action, location, status, t, battery):
36         if action == "Suck":
37             return 10 * time_factor(t) if status == "Dirty" else -0.5 # suckin
38         if action in ("Left", "Right"):
39             dest = "A" if action == "Left" else "B"
40             if dest == location:
41                 return -5.0 # a move that goes
42             u = p_dirty(dest, t) * 10 * time_factor(t + 1) - 2
43             if battery <= 2:
44                 u -= 5 # hoard energy whe

```

```

45         return u
46     return 0.0 # NoOp: the do-nothing action
47
48     def program(percept):
49         location, status = percept
50         state["t"] += 1
51         belief[location] = status
52         last_seen[location] = state["t"]
53         if state["battery"] <= 0:
54             return "NoOp"
55         scores = {a: utility(a, location, status, state["t"], state["battery"])
56                  for a in ("Suck", "Left", "Right", "NoOp")}
57         action = max(scores, key=scores.get)
58         if verbose:
59             pretty = " ".join(f"{a}={scores[a]:+6.1f}" for a in ("Suck", "Left", "Right", "NoOp"))
60             flag = "" if state["t"] <= deadline else " (past deadline)"
61             print(f" t={state['t']:2d} at {location} ({status:5s}) {pretty} {flag}")
62         if action == "Suck":
63             belief[location] = "Clean"
64         if action in ("Left", "Right", "Suck"):
65             state["battery"] -= 1
66         return action
67     return program
68
69
70 STEPS = 18
71 print(f"One run, deadline=6 – watch the agent give up as the clock runs out:\n")
72 env = StochasticVacuumEnvironment(seed=5, respawn_prob=0.15)
73 program = utility_agent_with_deadline(battery_start=STEPS, deadline=6, verbose=True)
74 for _ in range(STEPS):
75     env.execute(program(env.percept()))
76 print(f"\n -> performance={env.performance}, cleanliness={env.cleanliness():.0%}")
77
78 print(f"{'deadline':>10s}{'avg performance':>18s}{'avg cleanliness':>18s}")
79 print("-" * 46)
80 for deadline in (STEPS, 14, 12, 10, 8, 6, 4, 2, 0):
81     results = []
82     for s in range(60):
83         env = StochasticVacuumEnvironment(seed=s, respawn_prob=0.15)
84         program = utility_agent_with_deadline(battery_start=STEPS, deadline=deadline)
85         for _ in range(STEPS):
86             env.execute(program(env.percept()))
87         results.append((env.performance, env.cleanliness()))
88     print(f"{'deadline':>10d}{'sum(r[0] for r in results)/len(results):>18.1f}")

```

```
89         f"{sum(r[1] for r in results)/len(results):>17.0%}")
90
91     print("""
92     How the choices shift: early in the run a trip to the other square is worth
93     roughly p_dirty x 10 minus the -2 travel cost, so the agent patrols and cleans.
94     Once t passes the deadline, time_factor shrinks that payoff geometrically while
95     the travel cost stays at -2. The move term crosses below NoOp's flat 0 and the
96     agent stops travelling; a little later even Suck-on-dirt falls under 0 and it
97     stops cleaning altogether. Tighten the deadline and that crossover simply
98     happens sooner, so the table slides down gradually instead of flipping at one
99     value - cleanliness falls monotonically from 76% to 50%, and performance
100     tracks it (the 6-vs-4 blip is seed noise, not a real reversal).
101
102     That is what makes utility agents worth the extra machinery: nobody wrote
103     "if late, stop cleaning". The behaviour falls out of arithmetic between
104     competing terms. It is also why they are the hardest to debug - Exercise 1's
105     failure (parking on one square forever) was not a broken rule, it was two
106     terms priced wrong.""")
```

L1–14

The three changes, and a note that this cell reuses Exercise 1's stochastic world — a static world has no time dimension worth pressuring.

PYTHON

Fourteen comment lines. A block like this at the top of a cell is a normal way to record *why* the code below looks the way it does — something the code itself cannot say.

L16–17

Every knob is a parameter, so the behaviour can be swept rather than argued about.

PYTHON

A parameter list too long for one line, wrapped inside the parentheses. Python allows line breaks freely inside any bracket, so no continuation character is needed.

L18–20

State: battery and clock together, beliefs about each square, and when each was last observed. Squares start `"Unknown"`, which is distinct from clean or dirty.

PYTHON

Three dicts holding the agent's state. Line 19 builds one with a comprehension, giving every square the same starting value.

L22–24

The deadline term. A clean square is worth full value up to the deadline; afterwards its worth decays geometrically at `decay ** (t - deadline)`. At `decay=0.6` that is a fast fade — five steps past the deadline, value is down to about 8%.

PYTHON

`decay ** (t - deadline)` is exponentiation: `0.6 ** 3` is $0.6 \times 0.6 \times 0.6$. Combined with a conditional expression, the whole calculation fits on one line.

L26–33

The decaying belief. Known dirty is certainty (1.0); never visited is a coin flip (0.5); otherwise the probability it has become dirty again grows with how long ago you looked: `1 - (1 - respawn_belief) ** age`. This is Exercise 1's staleness fix expressed as a probability rather than a hard cutoff — and it is *calibrated to the true respawn rate*, which is what makes it a model rather than a guess.

PYTHON

Several `if` s each returning early, so by line 32 you know neither earlier case applied. Line 33's `1 - (1 - p) ** age` is the standard “probability it happened at least once in `age` tries”.

L35–37

`Suck` on dirt is worth 10, scaled by the deadline. On a clean square it now scores **−0.5** rather than 0 — a small explicit penalty that breaks the original's tie with `NoOp` and stops the agent burning battery on pointless sucking.

PYTHON

Line 37 is a conditional expression whose true-branch is itself a calculation. It reads:
`return 10 * time_factor(t) if the square is dirty, otherwise -0.5 .`

L38–42

The lookahead. A move is worth the probability of finding dirt, times its deadline-scaled value, minus the travel cost. Line 40 catches a move to the square you are already on. This is the line that makes travel rational — and it is only possible because line 26 gives the agent an opinion about a square it cannot see.

PYTHON

Line 39 picks a destination with another conditional expression. Line 42 stores an intermediate result in `u` so the next lines can adjust it before returning — easier to read than one enormous expression.

L43–45

The low-battery penalty, carried over unchanged.

PYTHON

Adjust the accumulated value, then return it.

L46

`NoOp` is still exactly 0 — the fixed baseline every other option is measured against. This is what makes the crossover meaningful.

PYTHON

A bare `return 0.0` reached only when none of the earlier branches returned — the fall-through case.

L48–52

Advance the clock, record the observation and its timestamp.

PYTHON

Reading and writing dict entries, including using a variable as the key.

L53–54

The hard battery constraint still overrides everything.

PYTHON

A guard clause returning early.

L55–57

Score all four actions into a dict, then take the best. `max(scores, key=scores.get)` returns the winning key.

PYTHON

Lines 55–56 are a dict comprehension whose values come from calling a function, spread across two lines inside the braces. Line 57's `max(scores, key=scores.get)` is a neat idiom. Looping over a dict gives its keys, and `scores.get` is the lookup method passed as a function without calling it — so this reads “of all the keys, give me the one whose value is largest”.

L58–61

Optional per-step tracing of every score. This is how you debug a utility agent: you cannot tell why it chose something without seeing the numbers it rejected.

PYTHON

`if verbose:` uses a boolean flag directly. Line 59 joins formatted strings with two spaces; `{scores[a]:+6.1f}` means always show the sign (+), pad to 6 characters, one decimal place, floating point. Line 61 nests quotes carefully: `state['t']` uses single quotes because the f-string itself uses double.

L62–66

Update belief after sucking, drain battery for real actions, return.

PYTHON

Two separate `if` s, so both can apply on the same step.

L70–76

One verbose run: 18 steps, deadline at 6, so you can watch the transition happen live.

PYTHON

Line 71 is an f-string with no placeholders, which is harmless. `\n` inserts a line break. Line 76's `{...:.0%}` formats a fraction as a whole-number percentage.

L78–89

The sweep: nine deadline settings $\times$ 60 seeds each. Sixty is not decoration — at 8 seeds the ordering was noisy enough to be misread as a real reversal.

PYTHON

A loop over a tuple of values, containing another loop over 60 seeds. Line 87 appends a tuple, so `results` becomes a list of pairs. Lines 88–89 are two adjacent f-strings joined automatically.

L91–106

The analysis, including an explicit note that the small 6-versus-4 wobble is seed noise, not a finding.

PYTHON

One long multi-line string.

OUTPUT

One run, deadline=6 – watch the agent give up as the clock runs out:

```
t= 1 at A (Clean) Suck= -0.5 Left= -5.0 Right= +3.0 NoOp= +0.0 -> Right
t= 2 at B (Clean) Suck= -0.5 Left= -0.5 Right= -5.0 NoOp= +0.0 -> NoOp
t= 3 at B (Clean) Suck= -0.5 Left= +0.8 Right= -5.0 NoOp= +0.0 -> Left
t= 4 at A (Clean) Suck= -0.5 Left= -5.0 Right= -0.5 NoOp= +0.0 -> NoOp
t= 5 at A (Clean) Suck= -0.5 Left= -5.0 Right= +0.8 NoOp= +0.0 -> Right
t= 6 at B (Clean) Suck= -0.5 Left= -1.1 Right= -5.0 NoOp= +0.0 -> NoOp
t= 7 at B (Clean) Suck= -0.5 Left= -1.0 Right= -5.0 NoOp= +0.0 -> NoOp (past
t= 8 at B (Clean) Suck= -0.5 Left= -1.2 Right= -5.0 NoOp= +0.0 -> NoOp (past
t= 9 at B (Dirty) Suck= +2.2 Left= -1.4 Right= -5.0 NoOp= +0.0 -> Suck (past
t=10 at B (Clean) Suck= -0.5 Left= -1.6 Right= -5.0 NoOp= +0.0 -> NoOp (past
t=11 at B (Clean) Suck= -0.5 Left= -1.7 Right= -5.0 NoOp= +0.0 -> NoOp (past
t=12 at B (Clean) Suck= -0.5 Left= -1.8 Right= -5.0 NoOp= +0.0 -> NoOp (past
t=13 at B (Dirty) Suck= +0.3 Left= -1.9 Right= -5.0 NoOp= +0.0 -> Suck (past
t=14 at B (Clean) Suck= -0.5 Left= -1.9 Right= -5.0 NoOp= +0.0 -> NoOp (past
t=15 at B (Clean) Suck= -0.5 Left= -2.0 Right= -5.0 NoOp= +0.0 -> NoOp (past
t=16 at B (Clean) Suck= -0.5 Left= -2.0 Right= -5.0 NoOp= +0.0 -> NoOp (past
t=17 at B (Clean) Suck= -0.5 Left= -2.0 Right= -5.0 NoOp= +0.0 -> NoOp (past
t=18 at B (Clean) Suck= -0.5 Left= -2.0 Right= -5.0 NoOp= +0.0 -> NoOp (past
```

-> performance=17, cleanliness=69%

deadline	avg performance	avg cleanliness
18	43.6	76%
14	41.6	74%
12	40.8	71%
10	38.2	67%
8	36.1	64%
6	34.6	61%
4	34.8	60%
2	33.7	57%
0	29.0	50%

How the choices shift: early in the run a trip to the other square is worth roughly $p_{\text{dirty}} \times 10$ minus the -2 travel cost, so the agent patrols and cleans. Once t passes the deadline, time_factor shrinks that payoff geometrically while the travel cost stays at -2. The move term crosses below NoOp's flat 0 and the agent stops travelling; a little later even Suck-on-dirt falls under 0 and it stops cleaning altogether. Tighten the deadline and that crossover simply happens sooner, so the table slides down gradually instead of flipping at one value - cleanliness falls monotonically from 76% to 50%, and performance tracks it (the 6-vs-4 blip is seed noise, not a real reversal).

That is what makes utility agents worth the extra machinery: nobody wrote "if late, stop cleaning". The behaviour falls out of arithmetic between

competing terms. It is also why they are the hardest to debug - Exercise 1's failure (parking on one square forever) was not a broken rule, it was two terms priced wrong.

What the trace shows

Follow the `Suck` column down the verbose run — the agent's own valuation of cleaning, decaying in real time:

STEP	VALUE OF SUCK ON DIRT	WHAT THE AGENT DOES
$t \leq 6$ (before deadline)	+10.0	Patrols: <code>Right</code> , <code>Left</code> , <code>Right</code> — travel is clearly worth it
$t = 9$	+2.2	Still sucks dirt under it, but no longer travels for it
$t = 13$	+0.3	Barely worth the actuation
$t \geq 14$	—	Idles. <code>NoOp</code> 's flat 0 now beats everything

The move column tells the same story from the other side: `Left` starts at +3.0 and sinks to -2.0 . Once it crosses below `NoOp` 's zero, travelling stops. A little later `Suck` crosses too, and cleaning stops.

The sweep

Tighten the deadline and the crossover simply happens earlier, so the table slides rather than flipping at some threshold — cleanliness falls monotonically from **76%** at no pressure to **50%** at maximum pressure.

WHY THIS IS THE PAYOFF EXERCISE

Nobody wrote “if late, stop cleaning.” There is no such branch anywhere in the code.

The behaviour emerges from arithmetic between competing terms — which is exactly what utility-based agents are for, and exactly why they are the hardest to debug.

Exercise 1's failure was not a broken rule; it was two terms priced wrong. A reflex agent's bugs are in its rules, where you can read them. A utility agent's bugs are in its *numbers*.

15 Where this lands

Across four exercises the same pattern keeps surfacing: **an architecture is not better or worse in the abstract — it is better or worse for an environment.**

ENVIRONMENT PROPERTY	WHAT IT BREAKS	WHAT IT REWARDS
Dirt reappears (non-episodic)	Beliefs that never expire	Re-sensing, or beliefs with a decay policy
Location not observable	Memoryless rules	Internal state — or randomisation as a cheap substitute
More than two squares	Fixed direction rules (the oscillation trap)	State for <i>direction</i> , not just contents
A deadline	Fixed goals	A utility function that can trade goals off

The through-line

Notice that every exercise found a bug the original six-step, two-square, fully observable episode could not surface:

- The **goal-based agent has no goal test** — hidden in the original because scoring 5 looked like a reasonable result rather than a missing branch.
- The **utility agent never travels** — hidden because it happened to start on the dirty square and scored top marks.
- The **model-based agent trusts stale beliefs** — hidden because in an episodic world beliefs cannot go stale.
- The **reflex agent oscillates at walls** — hidden because with two squares there is no wall to get stuck against.

The agents did not change. The environment got honest.

CARRY THIS INTO LECTURE 2

Most agent failures are environment assumptions that stopped holding. Not bad algorithms — correct algorithms, still running correctly, in a world that no longer matches the PEAS spec they were written against.

That is why serious agent work is dominated by *evaluation* rather than by architecture, and it is the same reason LLM agents are hard: the environment is the open world, and it never stops drifting away from the spec you built for.

If you want to go further

- 1 Combine Exercises 1 and 3: dirt that respawns in a row of eight. Does the decaying model-based agent still hold up, or does the row get too long to keep beliefs fresh?
- 2 Give the learning agent a reward that credits cleanliness rather than the `Suck` action, and see whether it stops parking.
- 3 Make movement stochastic — a 10% chance a move fails — and watch dead reckoning break. That is the step from a model to a *probabilistic* model, and it is where the next part of the course goes.

Every code block on this page was executed in order in a single Python 3.11 kernel, and every output block is the real stdout from that run — nothing here is transcribed by hand. To save this as a PDF, use your browser's Print command; the page has a print stylesheet that drops the sidebar and keeps code blocks from splitting across pages.